National Information Technology Development Agency

Folio Flow Package Guide

What a flow package is, how one is applied, the release history, and each package's changes

National Information Technology Development Agency · Office of the Director-General/CEO · DGO Digital Operations

Document	Folio Flow Package Guide
Reference	NITDA-FOLIO-HO-3.2
Version	1.0
Date	20 October 2026
Classification	Internal
Built from	Repository dgoeaa/ECM_DOCS_DEV, commit c6e49d73
Build	v2026.10.20-1 (Workspace and Portal packages)
Register of record	Version 1.12.87: 0 open, 122 closed

Contents

Flow packages 3

What a flow package is 3

Applying a package 3

The packages at release 41 3

Release history 6

Annex A · Document Portal and scan intake packages 12

Before pasting 12

How to apply 12

Changes 14

Phase 3 tenant tests (§14.3) 28

Annex B · Workspace and scheduled packages 29

Before pasting 29

Flow Configuration for the new lists 29

How to apply 30

Caller identity (G-04) 34

One bulk assignment endpoint 35

Changes 35

Flow packages

What a flow package is

A flow package is the definition a Folio flow runs, kept in the repository, tested offline, and released as a set. Folio has two kinds.

Kind	Flows	File	How it is applied
Identification package	27 Workspace, Portal and scheduled flows	`.paste.json` : the flow's root scope	Pasted into the existing flow in the designer, or written by `npm run flows:update -- apply` (Annexes A and B)
Import package	The Mailer's 7 flows	`.zip` : a Power Automate export	Imported into the tenant, as the Mailer's setup guide in 4.3 describes

A release freezes every identification package by its fingerprint. Release 41, of 19 Oct 2026, is the commissioned release. The paste ledger records, by fingerprint, what each flow in the tenant runs; at handover every package runs at its released version. A new release is cut only for a defect found in the tenant or an owner decision that Flow Configuration cannot meet, and its changes are released together.

Applying a package

1. Back the flow up: export it, or let `npm run flows:update -- apply` back it up for you.
2. Apply the package, as its annex describes for that flow: paste the root scope in place of the old one, keeping the flow's **Initialize variable** steps above it. If a step shows *Invalid connection*, choose the existing connection once.
3. Save, and run the flow's test.
4. Record what the flow now runs: `npm run flows:update -- check` reads every flow back and saves a report, and `npm run flows:update -- record <report>` files that report as evidence and updates the paste ledger.

The packages at release 41

Each package is in *3.3 Flow Packages*, in the folder named below. The fingerprint identifies the package's content; "In the tenant" compares it with what the paste ledger records the flow runs.

1 Folio Workspace

Package file	Flow	Fingerprint	In the tenant
ACTIVITY_LIFECYCLE__IP_Activity_Lifecycle_Endpoint__b4db02d8.paste.json	IP_Activity_Lifecycle_Endpoint	`0d2de979ee3e0b15`	Yes, at this version
AI_DOC_ANALYSIS__AI_Document_Processing__fefa43c6.paste.json	AI_Document_Processing	`196182c15102a70c`	Yes, at this version
BULK_ASSIGNMENT__BULK_OPS_DATA_RETRIEVAL_HTTP__5aab9e61.paste.json	BULK OPS DATA RETRIEVAL HTTP	`010c2e4f4b98771a`	Yes, at this version
DYNAMIC_ACTIONS__IP_Dynamic_Global_Actions_Endpoint__31c0aaf0.paste.json	IP_Dynamic_Global_Actions_Endpoint	`12b4e01076a45306`	Yes, at this version
EMAIL_RELATED_TASK__IP_Create_Task_for_received_Email_Endpoint__ab1a3555.paste.json	IP_Create__Task__for_received_Email_Endpoint	`12f2a35aac49e400`	Yes, at this version
EMAIL__IP_SEND_EMAIL__df59a69f.paste.json	IP_SEND_EMAIL	`10cbf77eec0fb118`	Yes, at this version
FETCH_ALL__IP_FETCH_ALL_BOOTSTRAP__b79e6707.paste.json	IP_FETCH_ALL_BOOTSTRAP	`01c2a838617f7903`	Yes, at this version
FETCH_EMAIL_ATTACHMENTS__IP_Retrieve_Email_Attachment_Endpoint__5bc87cf3.paste.json	IP_Retrieve_Email_Attachment_Endpoint	`1446a2b78727dc03`	Yes, at this version
GET_DOCS__IP_Get_Docs_Endpoint__28f6e62a.paste.json	IP_Get_Docs_Endpoint	`1a4ba31f5953e610`	Yes, at this version
OTP_GENERATE__IP_OTP_GENERATE__e5632336.paste.json	IP_OTP_GENERATE	`084639338fbcba0f`	Yes, at this version
OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde.paste.json	IP_OTP_VERIFY	`1c6655ae8be15503`	Yes, at this version
REFERENCE_DATA__IP_Reference_Data_Endpoint__9a4dea8e.paste.json	IP_Reference_Data_Endpoint	`178eac8c90727b19`	Yes, at this version
SCAN_INTAKE__IP_SCAN_INTAKE__5504d7f9.paste.json	IP_SCAN_INTAKE	`18a45281eccc8809`	Yes, at this version
SINGLE_ASSIGNMENT__Single_Task_Assignment__3caf9458.paste.json	Single_Task_Assignment	`08250b313d5af407`	Yes, at this version
USER_ADMIN__IP_User_Admin_Endpoint__e3e5c825.paste.json	IP_User_Admin_Endpoint	`1e9cbb576cfeb11f`	Yes, at this version

2 Document Portal

Package file	Flow	Fingerprint	In the tenant
STATUS__CG_Portal_Status_Check_Endpoint__60042a2e.paste.json	CG_Portal_Status_Check_Endpoint	`0ba3769c4e1c0e0e`	Yes, at this version
SUBMISSION__CG_Submission_Endpoint__de9ef13b.paste.json	CG_Submission_Endpoint	`1201c25e762f5c0b`	Yes, at this version
SUPPORT__CG_Portal_Support_Endpoint__4873c936.paste.json	CG_Portal_Support_Endpoint	`131612c75aa0f204`	Yes, at this version
UPLOAD__CG_Upload_Endpoint__df82e331.paste.json	CG_Upload_Endpoint	`03185bfbd7c73e1b`	Yes, at this version
VERIFY_CONFIRM__CG_Portal_Verification_Confirmation_Endpoint__f96e3448.paste.json	CG_Portal_Verification_Confirmation_Endpoint	`04fbe6443076bb11`	Yes, at this version
VERIFY__CG_Portal_Verification_Endpoint__ba47f79b.paste.json	CG_Portal_Verification_Endpoint	`17563b6f606b9309`	Yes, at this version
WRITEBACK__CG_Portal_Writeback_Endpoint__bde13d27.paste.json	CG_Portal_Writeback_Endpoint	`0912abac4b0d4b1b`	Yes, at this version

3 Scheduled and Event-Driven

Package file	Flow	Fingerprint	In the tenant
NOKEY__AUTO_SCHEDULED_SWEEP__8c19b45c.paste.json	AUTO_SCHEDULED_SWEEP	`04ec0c243abf2714`	Yes, at this version
NOKEY__CG_Portal_Daily_Sweep__f12aec6d.paste.json	CG_Portal_Daily_Sweep	`1856a501c435330d`	Yes, at this version
NOKEY__CG_Registry_Status_Notifier__8ebf6001.paste.json	CG_Registry_Status_Notifier	`01c3862f3e093d0f`	Yes, at this version
NOKEY__Folio_Failure_Watch__c38bfeaa.paste.json	Folio_Failure_Watch	`020035b509bc3d04`	Yes, at this version
NOKEY__Folio_Incoming_Correspondence_Scan_Processing_Flow__197527c8.paste.json	Folio_Incoming_Correspondence_Scan_Processing_Flow	`1c8bd444165b9411`	Yes, at this version

4 Correspondence Mailer

Package file	Flow	Fingerprint	In the tenant
DGO_MAIL_AUTH.zip	DGO_MAIL_AUTH	`68f85b89d14a16a9`	Yes (Mailer 2.1.5)

Package file	Flow	Fingerprint	In the tenant
DGO_MAIL_DATA.zip	DGO_MAIL_DATA	`55350896f628a71e`	Yes (Mailer 2.1.5)
DGO_MAIL_DIRECTORY.zip	DGO_MAIL_DIRECTORY	`9237bd3b4e27e5a4`	Yes (Mailer 2.1.5)
DGO_MAIL_LOG.zip	DGO_MAIL_LOG	`d9f953d7302c4335`	Yes (Mailer 2.1.5)
DGO_MAIL_PROVISION.zip	DGO_MAIL_PROVISION	`992ea3f847b2b6dc`	Yes (Mailer 2.1.5)
DGO_MAIL_SEND.zip	DGO_MAIL_SEND	`e90cb4ca67723a73`	Yes (Mailer 2.1.5)
DGO_MAIL_TEMPLATES.zip	DGO_MAIL_TEMPLATES	`fd4e2121e4cc829f`	Yes (Mailer 2.1.5)

Release history

Every release of the identification packages, newest first.

Release	Date	Packages changed	Reason
41	19 Oct 2026	2	Defect found on the deployment (2026-10-19): the host's browser check redirects without the #fragment, so a first visit through the digest, reminder or bulk-failure link to /#/acknowledgment opened the Command Center; the links now go to /acknowledge.html, which the Workspace routes to the acknowledgment queue (config/deeplink.config.js pathRoutes)
40	19 Oct 2026	1	New E-Scan notice to dgs@nitda.gov.ng: the owner set Send_an_email_(V2) To in the designer on 2026-10-19 (read back by the Flow API the same day); the package adopts it, with its delivery receipt
39	19 Oct 2026	21	Owner, 2026-10-19: the Workspace moved to folio-platform.page.gd and the Portal to folio-portal.page.gd (the rf.gd hosts are suspended). Every e-mail image and link, and every deep link, now points at the new hosts.
38	19 Oct 2026	8	Owner decision 2026-10-18 (build, not defer): RN-025 run-record e-mail in UPLOAD and WRITEBACK; RN-035 task reminders (DYNAMIC_ACTIONS upsert-record RMD to DGO_TaskReminders, AUTO_SCHEDULED_SWEEP sends them when due; the shared engine carries the RMD type into USER_ADMIN and ACTIVITY_LIFECYCLE); ITEM-91 directorate tasks in FETCH_ALL by the newly indexed GDSUROUT and DIRECTORATE_UNIT_MAP
37	18 Oct 2026	1	Owner decision 2026-10-18: the scan intake's New E-Scan notice goes to the registry mailbox (dgsregistry@nitda.gov.ng), not dgs@nitda.gov.ng; the administrative-recipient rule now covers a plain fixed value.

Release	Date	Packages changed	Reason
36	18 Oct 2026	21	ITEM-68 closure (owner, 2026-10-18): the owner-supplied brand template set (Comprehensive Portfolio Brand System Package) replaces all 43 catalogue templates: green masthead with the white NITDA lockup on every message, the DGO Digital Ops logo on internal mail, a greeting in every body, colour by purpose, logos served only from https://correspondence-gateway.rf.gd/email-assets/. Three off-palette colours mapped to the nearest brand tokens; two over-long expressions split with identical output; the failure-watch email given the same masthead.
35	18 Oct 2026	2	Tenant defect found by a commissioning support case (CG_Portal_Support_Endpoint run 08584092765524672908188781689CU23, 2026-10-18): the registry helpdesk alert used the citizen's address as Reply-To, and a malformed address made Outlook refuse the alert. Support and writeback registry alerts now use the citizen's address as Reply-To only when it is well formed.
34	18 Oct 2026	1	Tenant defect in Folio_Failure_Watch found on its proof run (run 08584092772023323576656283964CU24, 2026-10-18): the list returned the stored cursor reformatted ('10/18/2026 13:19:08'), so the new-failure window matched nothing. The cursor is now stored as utc:<ISO>; an unprefixed value is still parsed.
33	18 Oct 2026	1	Tenant defects in Folio_Failure_Watch found on its first live run (run 08584092775455011877820853803CU24, 2026-10-18): Select_Open_Failures refused substring() of an empty LastError; the run ended Succeeded although it could not classify. Fixed: empty-error guard, the run ends Failed when the watch cannot do its job, and the cursor does not move past a window whose report failed.
32	18 Oct 2026	1	Folio_Failure_Watch created in the tenant (flow c38bfeaa, 2026-10-18): its package takes the flow's id; the tenant refused an Hour recurrence with set hours, so the watch runs on a Week recurrence, Monday-Friday 07:00-18:00 WAT at :05 (hourly in working hours). Package content unchanged.
31	18 Oct 2026	5	Failure register and hourly watch (owner, 2026-10-18); legacy LIBRARY_INTAKE entries removed; fixes tenant defect AUTO_SCHEDULED_SWEEP run 08584093115454153780858604181CU30 (date-only list values refused by convertFromUtc, null digest recipients)
30	12 Oct 2026	2	Brand palette (2026-10-12): ACTIVITY_LIFECYCLE takes the shared failed-run alert template and AUTO_SCHEDULED_SWEEP's e-mails take palette colours only; the brand authority check found the colours extracted from the older captures.
29	12 Oct 2026	1	Tenant defect found 2026-10-12 (run 08584097726339433081123807376CU29): a task raised from an e-mail sent its assignee two near-identical notices; EMAIL_RELATED_TASK now sends one (Scope_3 removed).

Release	Date	Packages changed	Reason
28	10 Oct 2026	2	Owner decisions 2026-10-10. FETCH_ALL (ITEM-74, build an e-mail source): the registry tier receives the registry inbox's newest 50 messages (Get emails (V3), summary fields only) so Lookup can raise a task from an e-mail. AUTO_SCHEDULED_SWEEP (new package): one digest per person listing all their overdue or unacknowledged tasks, at most once a day, and every sweep e-mail goes to the registry until Flow Configuration NOTIFY_SWEEP_RECIPIENTS is staff (no staff e-mail before commissioning).
27	10 Oct 2026	1	Owner decision 2026-10-10 (ITEM-25, build them): ACTIVITY_LIFECYCLE, the new package for IP_Activity_Lifecycle_Endpoint (b4db02d8, created 2026-10-10 as a stopped copy of IP_Dynamic_Global_Actions_Endpoint), carrying only activity-archive, activity-siwes and activity-nysc in Folio's three steps, with only their own input rules (the service refuses one expression over 8,192 characters); DYNAMIC_ACTIONS has no room under the 500-action limit.
26	10 Oct 2026	7	Tenant defects found 2026-10-10 and owner directions of 2026-10-10. SINGLE_ASSIGNMENT and BULK_ASSIGNMENT: the task e-mail greeting falls through empty names (no 'Dear ,'), Response Priority no longer repeats Handling Status, and the Support Access link carries the Task Reference (TASK-2026-00002 e-mail). FETCH_ALL, GET_DOCS, REFERENCE_DATA: no 'Flow Run Record' e-mail of the reply payload, the run record keeps the reply's size only (ITEM-52, observed in the registry Sent folder). FETCH_ALL: comment author name and e-mail. DYNAMIC_ACTIONS: a task's own assignee may update it whatever their role, others 403; activity lifecycle steps refused as unknown (400), not 501 (ITEM-25; they move to their own flow, DYNAMIC_ACTIONS being at the 500-action limit). OTP_GENERATE: a code is created and mailed only to an active DGO_UserDirectory member, with the same reply for any address (owner direction 2026-10-10).
25	8 Oct 2026	1	USER_ADMIN failure alert e-mail takes the brand palette: its template was the captured page (#F6F8F6, #F0EFEF, outside the palette); it now shares the DYNAMIC_ACTIONS alert template, which differed only in those colours. Fixes test:brand, failing since 3f7012cd.
24	8 Oct 2026	1	USER_ADMIN: the service refused release 23's scope (an expression of 10,391 characters, limit 8,192): in IP_User_Admin_Endpoint the validation expression holds only the user-admin rules. flows:update now blocks an expression over the limit before writing.
23	8 Oct 2026	2	ITEM-25 (owner approval 2026-10-08): user-admin on its own flow, IP_User_Admin_Endpoint (USER_ADMIN, e3e5c825), created as a stopped copy of IP_Dynamic_Global_Actions_Endpoint; its package carries only user-admin on the same gate. DYNAMIC_ACTIONS refuses user-admin as unknown.

Release	Date	Packages changed	Reason
22	8 Oct 2026	1	ITEM-90 (owner approval 2026-10-08): FETCH_ALL admits every active directory user and scopes the reply by role: all tier (systemAdmin, userAdmin, executive, operator, admin/executive/registry persona) unchanged; everyone else their own tasks (by indexed AssignedTo), those tasks' documents and comments; directors also approvals; addresses only for the all tier and the caller's own row.
21	8 Oct 2026	1	Revert of release 20: the service refused the DYNAMIC_ACTIONS write (TemplateValidationError: 524 actions, limit 500), so nothing changed in the tenant. user-admin needs its own flow (owner approval pending: no new flow before commissioning); DYNAMIC_ACTIONS is back to release 19 as it runs live.
20	8 Oct 2026	1	ITEM-25 needs assessment: Folio's User Administration screen (user-admin:create-user, update-user, assign-role, disable-user) answered 501, so its changes never reached DGO_UserDirectory. DYNAMIC_ACTIONS now serves them, with the directory's own choices validated and systemAdmin granted only by a systemAdmin.
19	8 Oct 2026	1	ITEM-25 needs assessment: update-status and delete-record have no caller anywhere, so DYNAMIC_ACTIONS refuses them as unknown (400) rather than recognised-but-unserved (501).
18	8 Oct 2026	1	Owner decision 2026-10-08 (ITEM-25, on the needs assessment): dispatch, dispatchoutbound and archivereference are served by transitionstatus instead of answering 501: dispatch records the document Treated (the letter goes through the Mailer); archive records it Treated and sets IdState ARCHIVED. Routes dispatch and archive.
17	8 Oct 2026	15	Security defect found in the tenant (ITEM-30 live read): the run-record header redaction masked Authorization but not X-DGO-Proof, where the sign-in proof travels since release 9, so every telemetry row since 2026-10-08 15:54Z stored a usable session token. Every flow that records headers now masks it under both spellings.
16	8 Oct 2026	1	Tenant defect (ITEM-43 live test): IP_OTP_VERIFY answered 500 to a verify for an address with no code issued (Update_item_Attempts had no row). The count is written only when an unverified code exists; otherwise the flow answers as a wrong code does.
15	8 Oct 2026	3	Owner decision 2026-10-08 (folio-gateway.rf.gd primary): task e-mail links in SINGLE_ASSIGNMENT, BULK_ASSIGNMENT and EMAIL_RELATED_TASK start at the primary host, not taskack.page.gd; and the single and bulk task e-mail's Acknowledge Receipt and Support Access buttons, which linked nowhere (href empty), now carry their links.
14	8 Oct 2026	1	Owner decision 2026-10-08: folio-gateway.rf.gd is the Workspace's primary host. The scan-processing flow's View Document link (APP_FAST_TRACK_LINK) now starts there, read from config/deeplink.config.js productionBase instead of a literal.

Release	Date	Packages changed	Reason
13	8 Oct 2026	1	Write-back timeline rows carry their own Entity Key and name the submission by Qualified Key (RootKey, SubjectKey), as the RECEIVED row does (§10.2); found on the first live write-back.
12	8 Oct 2026	1	Tenant defect, first live write-back: SCAN_INTAKE read the directory Role choice as text, so the RoleId lookup never matched and every caller was refused (403 on write-back; no officer could deposit). It now reads the role as FETCH_ALL does (ROLE_OF, now shared).
11	8 Oct 2026	2	Tenant defect, first live write-back: substring() of an empty string fails in the service, so SCAN_INTAKE failed every request without X-DGO-Filename (all write-backs) and DYNAMIC_ACTIONS failed upsert-record for a titled row without text. Both now pad before cutting; the simulator now refuses substring('',0,0) as the service does.
10	8 Oct 2026	1	Tenant defect: FETCH_ALL read Global Tracking Queue (16,071 rows, 20 indexes, none on Modified) ordered by Modified, which exceeds the list view threshold; every read after it was skipped and the Workspace loaded empty. Tasks are now read by ID.
9	8 Oct 2026	11	Tenant defect found live 2026-10-08: on a signed (sig=) trigger Power Automate reads an Authorization header as an Entra token and refuses the call (401 MISE unauthorized) before the flow runs, so every signed-in Folio call failed and Folio loaded no data. The 11 gated packages read the sign-in proof from X-DGO-Proof first (Authorization kept as a fallback); Folio sends X-DGO-Proof. Written by API, no owner paste.
8	8 Oct 2026	1	Owner direction 2026-10-08 (proceed as recommended): D1 staff status write-back as a second branch on the intake flow IP_SCAN_INTAKE (X-DGO-Action writeback; officer from the sign-in proof; correspondence route; Portal Registry status and a timeline row). Also a tenant defect: X-DGO-Size was converted with int() in both arms of if(), so a request without it failed the scope. Only SCAN_INTAKE changes; written by API, no owner paste.
7	8 Oct 2026	2	Owner decisions 2026-10-08. SCAN_INTAKE: the trigger is opened to anyone like the other endpoints, so the caller is taken from Folio's sign-in proof instead of the tenant header any caller could send. OTP_VERIFY: Case_Generate removed (ITEM-82) after a Folio sign-in ran generation in IP_OTP_GENERATE and verification in IP_OTP_VERIFY. Both written by API; no owner paste.
6	8 Oct 2026	1	Defect: the scan-processing rebuild (197527c8, not yet live) carried the legacy flow's broken deep links — the retired Power Apps player host in APP_FAST_TRACK_LINK and a stray 'veggie ' suffix in Set_variable_Deeplink/_2. Fixed from patches/deeplinks; only SCAN_PROCESSING changes. Written by API, no owner paste.

Release	Date	Packages changed	Reason
5	8 Oct 2026	2	Owner directions, 2026-10-08: ITEM-81 DYNAMIC_ACTIONS harmonised with the hardened scope (409 IDEMPOTENCY_CONFLICT, 503 OUTCOME_UNKNOWN, length limits; no new actions); ITEM-80 SCAN_PROCESSING added, the legacy library intake rebuilt in Folio_Incoming_Correspondence_Scan_Processing_Flow. No other package changes.
4	7 Oct 2026	1	Owner decision and approval, 2026-10-07 (ITEM-82): OTP generation on its own flow; adds the IP_OTP_GENERATE package, created in the tenant from IP_OTP_VERIFY with Case_Verify removed. No other package changes.
3	7 Oct 2026	1	Release 3 (2026-10-07). DYNAMIC_ACTIONS only: release 2 was refused for an expression over 8192 characters (the lean claim's number repeated in its summary); the number is now computed once in Compose_Claim_<tag>_Number. 493 actions, longest expression 4721.
2	7 Oct 2026	11	Release 2 (2026-10-07). ITEM-64 (owner decision): the internal proof ledger is the new list DGO_SessionProofs (444102d6); the mint (OTP_VERIFY) and every verifying package name it. ITEM-79 (defect found in the tenant): DYNAMIC_ACTIONS was refused at 551 actions; its two claims use the lean claim (same 10 attempts) and the flow is 491. Q1 (owner): DYNAMIC_ACTIONS replaces Scope_Dynamic_Flow_Production_Hardened.
1	6 Oct 2026	23	Release 1: the packages as built after the owner's package release rule (2026-10-06): Set variable values held in Compose steps (defect found in the tenant export of IP_Create__Task__for_received_Email_Endpoint), configurable administrative recipients back on Flow Configuration.

Annex A · Document Portal and scan intake packages

IDENTIFICATION_SYSTEM.md §13, Phase 3. Each file is the flow's root scope as the tenant runs it, from the conformance package (identification-conformance) or, for IP_SCAN_INTAKE, which that package does not cover, from the tenant capture, with the Phase 3 edits below applied. IP_SCAN_INTAKE's root scope sets the variables its flow declares above it; keep those **Initialize variable** steps when pasting it. `tests/identification-phase3.test.mjs` runs every file in the strict simulator (`scripts/lib/wdl-sim.mjs`) against an in-memory SharePoint that keeps the lists' columns and the counter's ETag rule.

Before pasting

- Phase 1 is complete: the identity columns exist on Portal Registry, Portal Status Timeline and Portal Flow Telemetry, and Flow Configuration holds `ENVIRONMENT_CODE` (`prd`) and the `DATASET_*` rows (provisioning).
- Phase 2 is complete: the kit's claim passed the tenant exit test (identification-phase2).
- **One list setting, once.** The VOID row (owner decision Q2) writes Status `void`, which is not among Portal Registry's Status choices (`portal-field-spec.json`: received, validation, review, action-required, approved, declined, withdrawn). NEDMS → Portal Registry → *List settings* → **Status** → add the choice `void` → OK. Nothing else changes; the existing choices and the default stay as they are.
- **One value to confirm, for SUPPORT.** In Flow Configuration, `DATASET_RECORD_LINKS_LIST_ID` must hold `c3875803-8a9c-42d6-86b9-85e3f0ca5eef`, the DGO_RecordLinks list ID the Phase 1 completion run recorded. The provisioning template shipped it as "(set after the list is created)". If it still says that, the ABOUT link is not written (the case itself is unaffected).

How to apply

All at once: `npm run flows:update -- apply` writes every package below into its flow, backing each flow up first and reading it back (FLOWS_UPDATE.md). Or by hand, one flow at a time:

For each flow: open it, delete the root scope named below, select the step above where it was, choose **Paste an action**, paste the file, and save. If a step shows *Invalid connection*, open it, choose the existing SharePoint connection once, and the others pick it up. Then run the flow's test.

Flow	Flow ID	Root scope	Steps	File	In the tenant
CG_Submission_Endpoint	`de9ef13b-ae4c-42b0-9afa-20e71a180759`	`Scope_Submission_Flow`	217	`SUBMISSION__CG_Submission_Endpoint__de9ef13b.paste.json`	In the tenant (2026-10-19)
CG_Portal_Support_Endpoint	`4873c936-a2be-4726-8e69-426edb8f70e6`	`Scope_Support_Flow`	100	`SUPPORT__CG_Portal_Support_Endpoint__4873c936.paste.json`	In the tenant (2026-10-19)
CG_Portal_Status_Check_Endpoint	`60042a2e-dadb-4ad8-a672-38b4132d299f`	`Scope_Status_Flow`	72	`STATUS__CG_Portal_Status_Check_Endpoint__60042a2e.paste.json`	In the tenant (2026-10-19)
CG_Portal_Writeback_Endpoint	`bde13d27-73b3-4e1a-9687-eb289b039c7a`	`Scope_Writeback_Flow`	124	`WRITEBACK__CG_Portal_Writeback_Endpoint__bde13d27.paste.json`	In the tenant (2026-10-19)
CG_Upload_Endpoint	`df82e331-b800-4a8d-996e-d2b2ca846c77`	`Scope_Upload_Flow`	68	`UPLOAD__CG_Upload_Endpoint__df82e331.paste.json`	In the tenant (2026-10-19)
CG_Registry_Status_Notifier	`8ebf6001-89cd-49c9-a9b6-4dedb9a5768b`	`Scope_StatusNotify_Flow`	87	`NOKEY__CG_Registry_Status_Notifier__8ebf6001.paste.json`	In the tenant (2026-10-19)
CG_Portal_Daily_Sweep	`f12aec6d-1554-483d-a525-a96ddb60d06f`	`Scope_Sweep_Flow`	66	`NOKEY__CG_Portal_Daily_Sweep__f12aec6d.paste.json`	In the tenant (2026-10-19)

Flow	Flow ID	Root scope	Steps	File	In the tenant
CG_Portal_Verification_Endpoint	`ba47f79b-a831-49e8-b00b-3f8937c331df`	`Scope_Verify_Flow`	79	`VERIFY__CG_Portal_Verification_Endpoint__ba47f79b.paste.json`	In the tenant (2026-10-19)
CG_Portal_Verification_Confirmation_Endpoint	`f96e3448-3c79-4d42-b224-825258a552ba`	`Scope_VerifyConfirm_Flow`	69	`VERIFY_CONFIRM__CG_Portal_Verification_Confirmation_Endpoint__f96e3448.paste.json`	In the tenant (2026-10-19)
IP_SCAN_INTAKE	`5504d7f9-cc96-470c-be79-217582faf415`	`Scope_Scan_Flow`	270	`SCAN_INTAKE__IP_SCAN_INTAKE__5504d7f9.paste.json`	In the tenant (2026-10-19)

Changes

CG_Submission_Endpoint

Source: the identification-conformance package (`SUBMISSION__CG_Submission_Endpoint__de9ef13b.paste.json`).

- The proof check is safe when no proof is sent. The portal's first call never carries one, so the proof lookup is skipped and its value is null; first() of null fails the step, which the live flow escapes only if the service stops evaluating and() at its first false argument. It now reads first() of an empty list instead.
- B5 / §8.3: duplicate check by query, then comparison. The register is queried for the same sender, subject and attachment count in the last 24 hours (indexed columns, as before), and each row found is compared on EventDate and AttachmentHashSet: the first 12 hex characters of each declared SHA-256, compared as a set (union sizes), so the order the files were declared in does not matter. The client localId is no longer read (§8.1, V9), and LocalId is no longer written (§10.2: retired).
- §8.2: the Registry Reference is claimed by the generation kit (10 flat attempts with a 1 to 5 s Delay, HTTP MERGE with IF-MATCH, the counter row found by Prefix and Year and created by the first claim of a year), as proven by the Phase 2 exit run. The flow's own three attempts are removed. The kit reads the counter and register addresses from Flow Configuration (DATASET_SEQUENCE_COUNTERS_*, DATASET_INTAKE_REGISTER_*) and the environment prefix from ENVIRONMENT_CODE (prd: none). The claim never issues at or below the highest NITDA- number already in Portal Registry (the kit floor read): a counter row that is missing or

behind the register can no longer reissue a reference in use, as it did live on 5 October 2026 (NITDA-2026-00001 refused as a duplicate Reference ID).

- §10.2 / §6.5: the register row carries its Entity Key, IdState ACTIVE, Rev 1, Direction In, AttachmentHashSet, and the Request ID as both CorrelationId (this flow starts the transaction) and CreatedByRequest.
- §8.2 / B8: if the record write fails after the claim, a VOID row accounts for the number (Title VOID, ReferenceId, Status void, SenderEmail, IdState VOID, VoidReason RECORD_WRITE_FAILED: owner decision Q2). The run still ends in the 500 path.
- §10.2 STATUS_TIMELINE: the RECEIVED event carries its own Entity Key, SubjectKey and RootKey (REC:), RootRef, Action RECEIVED, RequestId, CorrelationId, RevAfter 1, Process INTAKE and ContractKey SUBMISSION.
- §8.3: every reply carries requestId, the flow's own guid().
- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO:, and ContractKey SUBMISSION.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Registry_Alert_Email, Send_Sender_Acknowledgement_Email, Send_Rate_Limit_Alert_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`, `Compose_EntityKey_Record`, `Compose_Claim_Registry_Year`, `Get_Claim_Registry_Floor`, `Compose_Claim_Registry_Floor`, `Compose_Claim_Registry_Count`, `Scope_Claim_Registry_1`, `Get_Counter_Registry_1`, `Condition_Counter_Registry_1_Exists`, `Compose_Next_Registry_1`, `Claim_Counter_Registry_1`, `Compose_Claimed_Registry_1`, `Create_Counter_Registry_1`, `Compose_Created_Registry_1`, `Scope_Claim_Registry_2`, `Delay_Claim_Registry_2`, `Get_Counter_Registry_2`, `Condition_Counter_Registry_2_Exists`, `Compose_Next_Registry_2`, `Claim_Counter_Registry_2`, `Compose_Claimed_Registry_2`, `Create_Counter_Registry_2`, `Compose_Created_Registry_2`, `Scope_Claim_Registry_3`, `Delay_Claim_Registry_3`, `Get_Counter_Registry_3`, `Condition_Counter_Registry_3_Exists`, `Compose_Next_Registry_3`, `Claim_Counter_Registry_3`, `Compose_Claimed_Registry_3`, `Create_Counter_Registry_3`, `Compose_Created_Registry_3`, `Scope_Claim_Registry_4`, `Delay_Claim_Registry_4`, `Get_Counter_Registry_4`, `Condition_Counter_Registry_4_Exists`, `Compose_Next_Registry_4`, `Claim_Counter_Registry_4`, `Compose_Claimed_Registry_4`, `Create_Counter_Registry_4`, `Compose_Created_Registry_4`, `Scope_Claim_Registry_5`, `Delay_Claim_Registry_5`, `Get_Counter_Registry_5`, `Condition_Counter_Registry_5_Exists`, `Compose_Next_Registry_5`, `Claim_Counter_Registry_5`, `Compose_Claimed_Registry_5`, `Create_Counter_Registry_5`, `Compose_Created_Registry_5`, `Scope_Claim_Registry_6`, `Delay_Claim_Registry_6`, `Get_Counter_Registry_6`, `Condition_Counter_Registry_6_Exists`,

`Compose_Next_Registry_6`, `Claim_Counter_Registry_6`, `Compose_Claimed_Registry_6`, `Create_Counter_Registry_6`, `Compose_Created_Registry_6`, `Scope_Claim_Registry_7`, `Delay_Claim_Registry_7`, `Get_Counter_Registry_7`, `Condition_Counter_Registry_7_Exists`, `Compose_Next_Registry_7`, `Claim_Counter_Registry_7`, `Compose_Claimed_Registry_7`, `Create_Counter_Registry_7`, `Compose_Created_Registry_7`, `Scope_Claim_Registry_8`, `Delay_Claim_Registry_8`, `Get_Counter_Registry_8`, `Condition_Counter_Registry_8_Exists`, `Compose_Next_Registry_8`, `Claim_Counter_Registry_8`, `Compose_Claimed_Registry_8`, `Create_Counter_Registry_8`, `Compose_Created_Registry_8`, `Scope_Claim_Registry_9`, `Delay_Claim_Registry_9`, `Get_Counter_Registry_9`, `Condition_Counter_Registry_9_Exists`, `Compose_Next_Registry_9`, `Claim_Counter_Registry_9`, `Compose_Claimed_Registry_9`, `Create_Counter_Registry_9`, `Compose_Created_Registry_9`, `Scope_Claim_Registry_10`, `Delay_Claim_Registry_10`, `Get_Counter_Registry_10`, `Condition_Counter_Registry_10_Exists`, `Compose_Next_Registry_10`, `Claim_Counter_Registry_10`, `Compose_Claimed_Registry_10`, `Create_Counter_Registry_10`, `Compose_Created_Registry_10`, `Compose_Claim_Registry`, `Select_Claim_Registry_References`, `Create_Void_Registry`, `Select_Submission_Attachment_Hashes`, `Compose_Submission_Attachment_Hash_Set`, `Select_Submission_Duplicate_Matches`, `Compose_Submission_Duplicate_Reference`, `Compose_Sender_Acknowledgement_Html_Part4`, `Create_Send_Sender_Acknowledgement_Email_Receipt`, `Compose_Send_Sender_Acknowledgement_Email_Outcome`, `Compose_Registry_Alert_Html_Part4`, `Create_Send_Registry_Alert_Email_Receipt`, `Compose_Send_Registry_Alert_Email_Outcome`, `Create_Send_Rate_Limit_Alert_Email_Receipt`, `Compose_Send_Rate_Limit_Alert_Email_Outcome`, `Compose_Server_Error_Alert_Html_Part4`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: `Compose_Submission_LocalId`, `Compose_Submission_Year`, `Get_Sequence_Counter`, `Condition_Sequence_Counter_Exists`, `Create_Sequence_Counter`, `Get_Sequence_Counter_Current_1`, `Compose_Submission_Next_Sequence_1`, `Compose_Submission_Reference_1`, `Claim_Sequence_Number_1`, `Get_Sequence_Counter_Current_2`, `Compose_Submission_Next_Sequence_2`, `Compose_Submission_Reference_2`, `Claim_Sequence_Number_2`, `Get_Sequence_Counter_Current_3`, `Compose_Submission_Next_Sequence_3`, `Compose_Submission_Reference_3`, `Claim_Sequence_Number_3`.

Test after pasting. Submit once from the portal with an attachment. Expect a NITDA-YYYY-NNNNN reference and a requestId in the reply; in Portal Registry the new row has EntityKey, IdState ACTIVE, Rev 1, CorrelationId and AttachmentHashSet, and no LocalId; in Portal Status Timeline the RECEIVED row has RootRef = the reference. Submit the same form again within 24 hours: the reply is the same reference with replayed: true, and no counter number is used.

CG_Portal_Support_Endpoint

Source: the identification-conformance package (`SUPPORT__CG_Portal_Support_Endpoint__4873c936.paste.json`).

- Send_Helpdesk_Alert_Email: the citizen's address is its Reply-To only when well formed; otherwise the registry's reply address (a malformed address made Outlook refuse the alert, tenant 2026-10-18).

- §8.3: the live draw (five CASE-XXXXXX candidates checked in one query, the first unused one taken) already meets the case reference rule and is kept. When all five collide, the flow now answers 503 RETRY_LATER and writes nothing; before, it wrote the fifth candidate, a duplicate. The replay branch it replaces was unreachable: no client reference is accepted (§8.1).
- A repeat of the same case is answered with the case already opened, not a second one: a case from the same e-mail address with the same topic and message in the last hour returns its reference (replayed: true), writes nothing and sends no e-mail. On 5 October 2026 the endpoint ran 24 times in one minute for one case from one phone (the portal outbox resending it; fixed in document-portal/js/core.js), and a browser that gives up on a slow run resends a case the flow did create. Portal Support Cases holds under a hundred rows, so the address and time filter is safe without an index; the message, a Note column, is compared in the flow. A failed lookup counts as no repeat.
- §10.2 SUPPORT_CASES: the case row carries its Entity Key, CaseRef, IdState ACTIVE, Rev 1, and the Request ID as CorrelationId and CreatedByRequest.
- §6.4 / T5: when the request names a reference, the flow finds that record by exact ReferenceId (V10) and, if the record has an Entity Key, writes an ABOUT link CAS → REC to Record Links (LinkMatchKey = FromKey|ABOUT|ToKey), addressed through Flow Configuration (DATASET_RECORD_LINKS_*). A record written before Phase 3 has no Entity Key until the Phase 5 backfill; it keeps the free-text AboutReference only. A failed link never fails the case: the reply follows the link step whatever its outcome.
- §8.3: every reply carries requestId, the flow's own guid().
- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO:, and ContractKey SUPPORT.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Helpdesk_Alert_Email, Send_Support_Acknowledgement_Email, Send_Rate_Limit_Alert_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Condition_Support_Collision`, `Condition_Support_Repeat`, `Compose_Result_Support_200_Repeat`, `Compose_Result_Support_503`, `Compose_EntityKey_Case`, `Scope_About_Link`, `Condition_About_Given`, `Get_About_Record`, `Condition_About_Has_Key`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_About_Link`, `Create_About_Link`, `Get_Recent_Support_Cases`, `Select_Support_Repeat`, `Compose_Support_Repeat_Ref`, `Compose_Support_Acknowledgement_Html_Part3`, `Create_Send_Support_Acknowledgement_Email_Receipt`, `Compose_Send_Support_Acknowledgement_Email_Outcome`, `Compose_Helpdesk_Alert_Html_Part3`, `Create_Send_Helpdesk_Alert_Email_Receipt`, `Compose_Send_Helpdesk_Alert_Email_Outcome`,

`Create_Send_Rate_Limit_Alert_Email_Receipt`, `Compose_Send_Rate_Limit_Alert_Email_Outcome`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: `Condition_Support_Replay`, `Compose_Result_Support_200_Replay`.

Test after pasting. Open a support case from the portal naming a reference issued after the SUBMISSION paste. Expect a CASE-XXXXXX case reference and a requestId; in Portal Support Cases the row has EntityKey, CaseRef = the case reference, IdState ACTIVE and Rev 1; in DGO_RecordLinks one ABOUT row whose FromKey is CAS:<the case's EntityKey> and ToKey is REC:<the record's EntityKey>. A case naming an older reference is created with AboutReference only and no link.

CG_Portal_Status_Check_Endpoint

Source: the identification-conformance package (`STATUS__CG_Portal_Status_Check_Endpoint__60042a2e.paste.json`).

- V10: the unread padded-reference step (Compose_Status_Ref_Padded) is removed, so no padded variant of a reference is ever computed.
- §8.2: a VOID row (a number claimed for a submission whose record write failed, so never issued to anyone) answers the same 404 as no record. The public endpoint gives a caller nothing about a number they were never given.
- §8.3: every reply carries requestId, the flow's own guid().
- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO:, and ContractKey STATUS.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: `Compose_Status_Ref_Padded`.

Test after pasting. Track a reference issued after the SUBMISSION paste, and a portal tracking ID from an older receipt, each with its own sender email. Both answer 200 with the record and a requestId; NITDA-2026-217 and NITDA-2026-00217 each resolve only to their own record (T6).

CG_Portal_Writeback_Endpoint

Source: the identification-conformance package (`WRITEBACK__CG_Portal_Writeback_Endpoint__bde13d27.paste.json`).

- Send_Writeback_Alert_Email: the citizen's address is its Reply-To only when well formed; otherwise the registry's reply address (a malformed address made Outlook refuse the alert, tenant 2026-10-18).
- V10: the unread padded-reference step (Compose_Writeback_Ref_Padded) is removed, so no padded variant of a reference is ever computed.
- §8.2: a VOID row is treated as no record (404), as in STATUS.
- B9 / §8.3: the register row is changed by the kit's Guarded Update (HTTP MERGE with IF-MATCH, Rev + 1, up to 3 attempts that each re-read the row), never by Update item, which cannot send IF-MATCH. The same columns change as before (UpdatedAtUtc, ClosedAtUtc, StatusLabel, ActionRequired, Status). If every attempt loses to a concurrent change, the reply is 503 RETRY_LATER and no timeline event is written.
- §10.2 STATUS_TIMELINE / ID-20: the writeback event carries its own Entity Key, SubjectKey and RootKey (REC:, when the record has one), RootRef, Action (WITHDRAWN, NOTE_ADDED or RESPONDED), RequestId, the record's CorrelationId (server state; the flow's own Request ID for a record written before Phase 3), RevAfter, Process INTAKE and ContractKey WRITEBACK.
- §8.3: every reply carries requestId, the flow's own guid().
- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO:, and ContractKey WRITEBACK.
- RN-025 (owner, 2026-10-18): the run record is e-mailed to the registry mailbox after the telemetry row (Send_Telemetry_Email, "CG_Portal_Writeback_Endpoint — Flow Run Record — "), with the record attached, as IP_SCAN_INTAKE does.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Writeback_Alert_Email, Send_Citizen_Respond_Email, Send_Citizen_Note_Email, Send_Citizen_Withdraw_Email, Send_Server_Error_Alert_Email, Send_Telemetry_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Scope_Update_Registry_1`, `Get_Row_Registry_1`, `Merge_Row_Registry_1`, `Compose_Rev_Registry_1`, `Scope_Update_Registry_2`, `Get_Row_Registry_2`, `Merge_Row_Registry_2`, `Compose_Rev_Registry_2`, `Scope_Update_Registry_3`, `Get_Row_Registry_3`, `Merge_Row_Registry_3`, `Compose_Rev_Registry_3`, `Compose_Updated_Registry`, `Condition_Writeback_Updated`, `Compose_Result_Writeback_503_Update`, `Create_Send_Citizen_Respond_Email_Receipt`, `Compose_Send_Citizen_Respond_Email_Outcome`, `Create_Send_Citizen_Note_Email_Receipt`, `Compose_Send_Citizen_Note_Email_Outcome`, `Compose_Citizen_Withdraw_Html_Part3`, `Create_Send_Citizen_Withdraw_Email_Receipt`, `Compose_Send_Citizen_Withdraw_Email_Outcome`, `Create_Send_Writeback_Alert_Email_Receipt`, `Compose_Send_Writeback_Alert_Email_Outcome`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`, `Compose_Telemetry_Attachments`,

`Send_Telemetry_Email`, `Create_Send_Telemetry_Email_Receipt`, `Compose_Send_Telemetry_Email_Outcome`.

Steps removed: `Compose_Writeback_Ref_Padded`, `Update_Registry_Record`.

Test after pasting. Add a note to a test submission from track.html. Expect 200; in Portal Registry the row's Rev went up by 1 and UpdatedAtUtc changed; in Portal Status Timeline the new row has Action NOTE_ADDED, RevAfter = the new Rev and RootRef = the reference.

CG_Upload_Endpoint

Source: the identification-conformance package (`UPLOAD__CG_Upload_Endpoint__df82e331.paste.json`).

- §10.2 ATTACHMENTS / §8.3: the attachment row carries its Entity Key, ParentKey and RootKey REC:<the record's key> (found by the ticket's SubmissionRef, exactly), RootRef, IdState ACTIVE, Rev 1 and CreatedByRequest. The Entity Key begins with the 8 hex characters SUBMISSION put in the stored file name, so the name carries the first 8 hex of the attachment's key as §8.3 requires; a ticket from before Phase 3 gets a fresh guid(). A record written before Phase 3 has no key, so ParentKey and RootKey stay empty until the Phase 5 backfill.
- §8.3: every reply carries requestId, the flow's own guid().
- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO:, and ContractKey UPLOAD.
- ID-20: UPLOAD continues the submission's transaction, so its telemetry CorrelationId is the record's, read from the register (server state), never from the browser. Without a record it is the flow's own Request ID.
- RN-025 (owner, 2026-10-18): the run record is e-mailed to the registry mailbox after the telemetry row (Send_Telemetry_Email, "CG_Upload_Endpoint — Flow Run Record — "), with the record attached, as IP_SCAN_INTAKE does.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Server_Error_Alert_Email, Send_Telemetry_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Get_Upload_Record`, `Compose_EntityKey_Attachment`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`, `Compose_Telemetry_Attachments`, `Send_Telemetry_Email`, `Create_Send_Telemetry_Email_Receipt`, `Compose_Send_Telemetry_Email_Outcome`.

Steps removed: none.

Test after pasting. Submit from the portal with one attachment (after the SUBMISSION paste). In Portal Attachments the new row has EntityKey beginning with the 8 hex characters in its StoredName, ParentKey and RootKey REC:<the register row's EntityKey>, RootRef = the reference, IdState ACTIVE and Rev 1; the telemetry row's CorrelationId equals the register row's CorrelationId.

CG_Registry_Status_Notifier

Source: the identification-conformance package (`NOKEY__CG_Registry_Status_Notifier__8ebf6001.paste.json`).

- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO: (no Contract Key: no endpoint calls this flow).
- §6.5 / ID-20: the notifier is triggered by the register row, so its CorrelationId is that row's (server state); a row written before Phase 3 has none and the run's own Request ID is used.
- Tenant defect 2026-10-18 (AUTO_SCHEDULED_SWEEP run 08584093115454153780858604181CU30): a SharePoint date-only column comes back without a zone, and convertFromUtc refused it. Every date this package converts from a list row, a trigger body or a connector reply (24) is read with a Z added when it has none, and an empty one reads as the epoch.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Status_Acknowledged_Email, Send_Status_Progress_Email, Send_Status_ActionRequired_Email, Send_Status_Approved_Email, Send_Status_Declined_Email, Send_Status_Withdrawn_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Compose_Status_Acknowledged_Html_Part5`, `Create_Send_Status_Acknowledged_Email_Receipt`, `Compose_Send_Status_Acknowledged_Email_Outcome`, `Compose_Status_Progress_Html_Part5`, `Create_Send_Status_Progress_Email_Receipt`, `Compose_Send_Status_Progress_Email_Outcome`, `Create_Send_Status_ActionRequired_Email_Receipt`, `Compose_Send_Status_ActionRequired_Email_Outcome`, `Create_Send_Status_Approved_Email_Receipt`, `Compose_Send_Status_Approved_Email_Outcome`, `Compose_Status_Declined_Html_Part4`, `Create_Send_Status_Declined_Email_Receipt`, `Compose_Send_Status_Declined_Email_Outcome`, `Compose_Status_Withdrawn_Html_Part3`, `Create_Send_Status_Withdrawn_Email_Receipt`, `Compose_Send_Status_Withdrawn_Email_Outcome`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: none.

Test after pasting. Change the Status of a test row created after the SUBMISSION paste. The e-mail arrives as before; the telemetry row has CorrelationId = the register row's CorrelationId and RegistryKey DGO:<the flow's ID>.

CG_Portal_Daily_Sweep

Source: the identification-conformance package (`NOKEY__CG_Portal_Daily_Sweep__f12aec6d.paste.json`).

- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId, ClientCorrelationId and the Flow Registry Key DGO: (no Contract Key: no endpoint calls this flow).
- ID-20: a scheduled sweep starts its own transaction, so its CorrelationId is its own Request ID. It writes no governed row (it marks upload tickets expired, which are not governed).
- Tenant defect 2026-10-18 (AUTO_SCHEDULED_SWEEP run 08584093115454153780858604181CU30): a SharePoint date-only column comes back without a zone, and convertFromUtc refused it. Every date this package converts from a list row, a trigger body or a connector reply (13) is read with a Z added when it has none, and an empty one reads as the epoch.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Expired_Upload_Email, Send_Registry_Digest_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Compose_Expired_Upload_Html_Part3`, `Create_Send_Expired_Upload_Email_Receipt`, `Compose_Send_Expired_Upload_Email_Outcome`, `Create_Send_Registry_Digest_Email_Receipt`, `Compose_Send_Registry_Digest_Email_Outcome`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: none.

Test after pasting. Run the flow once by hand. It ends Succeeded as before, and its telemetry row carries RequestId, CorrelationId (the same value) and RegistryKey.

CG_Portal_Verification_Endpoint

Source: the identification-conformance package (`VERIFY__CG_Portal_Verification_Endpoint__ba47f79b.paste.json`).

- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Verification_Code_Email, Send_Undelivered_Alert_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Create_Send_Verification_Code_Email_Receipt`, `Compose_Send_Verification_Code_Email_Outcome`, `Create_Send_Undelivered_Alert_Email_Receipt`, `Compose_Send_Undelivered_Alert_Email_Outcome`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: none.

Test after pasting. Request a verification code from the portal for a test address. The code arrives as before; Portal Outbox Receipts has a row with MessageType verification-code, Status sent and that address, and no code in any column.

CG_Portal_Verification_Confirmation_Endpoint

Source: the identification-conformance package (`VERIFY_CONFIRM__CG_Portal_Verification_Confirmation_Endpoint__f96e3448.paste.json`).

- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Lockout_Alert_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Create_Send_Lockout_Alert_Email_Receipt`, `Compose_Send_Lockout_Alert_Email_Outcome`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: none.

Test after pasting. Enter a wrong code six times for a test address. The lockout alert arrives as before and Portal Outbox Receipts has a row with MessageType lockout-alert, Status sent.

IP_SCAN_INTAKE

Source: docs/folio-Master-Flows-and-Endpoints/captures/IP_SCAN_INTAKE__5504d7f9-cc96-470c-be79-217582faf415.json (the tenant capture; this flow is not in the conformance package).

- §8.2: the year is no longer read in UTC and the LockToken is retired. Compose_Scan_Year and Compose_Scan_LockToken are removed; the kit's claim takes the WAT year.
- G-04, owner decision 2026-10-08: the trigger is open to anyone, as every other endpoint is, so the caller can no longer come from x-ms-client-principal-name (set only for a tenant-authenticated trigger; on an open one any caller could send it). The caller is the address of the sign-in proof Folio sends (core/scan-intake-service.js authHeaders), looked up in the proof ledger: unconsumed, Purpose internal-session, unexpired. A deposit always needs a valid proof: with none the caller is empty, no directory row matches, and the existing check answers 401. The directory and role check that follows is unchanged.

- §8.2: the Registry Reference is claimed by the generation kit (10 attempts, IF-MATCH, counter row found by Prefix and Year, floored at the highest number in Portal Registry), the same block SUBMISSION uses, so portal and scan share one number line (T2). The LockToken claim is removed. It also removes padLeft(), which the service refused live on 28 September ("The template function 'padLeft' is not defined or not valid"): every deposit that reached the number failed. When every attempt loses, the reply is 503 RETRY_LATER (was 409).
- S7 / §10.2: every scan now has a Portal Registry row, as every portal submission does: Title and ReferenceId = the reference, Subject = the file name, Channel scan, CorrespondenceType Incoming, Status received, AttachmentCount 1, AttachmentHashSet (first 12 hex of the declared SHA-256), EntityKey, IdState ACTIVE, Rev 1, Direction In, CorrelationId and CreatedByRequest = the Request ID, and SourceLocator = the stored file's path. SenderEmail is the registry mailbox (Flow Configuration NOTIFY_REGISTRY_TO, else dgsRegistry@nitda.gov.ng), owner decision 4 October 2026: a paper letter's sender is not known to the system, so status e-mails for scans go to the registry. The scanning officer stays on the audit event. The row is written before the file, and if it fails a VOID row accounts for the number (Q2).
- §10.2 SCAN_LIBRARY: the stored file's library item gets its identity columns (EntityKey, whose first 8 hex are in the file name; ParentKey and RootKey REC:<the record's key>; RootRef; IdState ACTIVE; Rev 1; CreatedByRequest) through the kit's Guarded Update, addressed through Flow Configuration (DATASET_SCAN_LIBRARY_*). A failure here does not undo the deposit: the reply follows whatever its outcome.
- The reply now carries the fields Folio's scan service reads at its top level (core/scan-intake-service.js: referenceId, attachmentLink, stored, depositedBy, depositedAt, sha256, bytes). The flow answered with an envelope whose data held reference and resource, so even a successful deposit showed the officer an empty reference. The envelope keeps its own fields.
- §10.2 AUDIT_LOG: the deposit event in DGO_AuditLog carries its Entity Key, SubjectKey and RootKey REC:, RootRef, CorrelationId, RevAfter 1, Process INTAKE and ContractKey SCAN_INTAKE.
- Two expressions carried over from the live flow are corrected. The notification condition tested contains(caller, "@") with a bare "@", which the workflow language reads as an empty expression, not the character; it is now "@@", the literal @. Compose_Complete_Run_Record wrapped its expression in @{...}, which makes the value text, so the final telemetry update's setProperty() on it failed on every run; it is now the object itself.
- §10.2 FLOW_TELEMETRY: the run row carries RequestId, CorrelationId (this flow starts the transaction), the Flow Registry Key DGO: and ContractKey SCAN_INTAKE.
- X-DGO-Size is read without failing when it is absent: the service evaluates both arms of if(), so int() of an empty header failed the scope before any check (a write-back carries no file). The value is chosen first and converted once: int(if(valid, the header, '0')); a deposit with no size still answers 400 from the size check.
- Tenant defect (8 October 2026, first live write-back): the caller's role is read from the directory row's Role choice, as FETCH_ALL reads it. The connector returns a Choice column as an object, which the flow read as text, so the RoleId lookup never matched and every caller was refused: 403 on a write-back, and no officer could deposit.

- Tenant defect (8 October 2026, first live write-back): an absent X-DGO-Filename failed the scope before any check, because substring() of an empty string fails ("start index must be less than the length") and a write-back carries no file. The decoded name is now empty when the header is: the name is padded with one character before it is cut and the pad is never part of the result. A deposit with no name still fails the metadata check.
- D1 / ITEM-6 (owner direction 2026-10-08, "proceed as recommended"): the staff status write-back is a second branch on the intake flow, which the owner identifies as IP_SCAN_INTAKE (ITEM-86). A request carrying X-DGO-Action: writeback sets a portal submission's status from Folio; every other request takes the deposit path unchanged. The caller is the sign-in proof's officer (above), who must be an active directory user (401) whose role carries "*" or the correspondence route (403). The body names referenceId, status (received, validation, review, action-required, approved or declined: the portal's governed values; withdrawn stays the citizen's) and optionally label and note (400 otherwise). The Portal Registry row is found by ReferenceId (404 if none) and gets Status, StatusLabel, ActionRequired, UpdatedAtUtc, AcknowledgedAtUtc on its first move past received, and ClosedAtUtc on approved or declined; a Portal Status Timeline row records the change with the officer as Actor. CG_Registry_Status_Notifier, triggered by the row change, tells the citizen.
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_varData_Scan_AuditFailed, Set_variable_varData_Scan_BadRequest, Set_variable_varData_Scan_Catch, Set_variable_varData_Scan_Denied_401, Set_variable_varData_Scan_Denied_403, Set_variable_varData_Scan_Deposited, Set_variable_varData_Scan_Duplicate, Set_variable_varData_Scan_Lock, Set_variable_varData_Scan_Oversize, Set_variable_varData_Scan_RateLimited, Set_variable_varStatusCode_Scan_409, Set_variable_varStatusCode_Scan_Writeback_200, Set_variable_varStatusCode_Scan_Writeback_400, Set_variable_varStatusCode_Scan_Writeback_401, Set_variable_varStatusCode_Scan_Writeback_403, Set_variable_varStatusCode_Scan_Writeback_404, Set_variable_varStatusCode_Scan_Writeback_500.
- Administrative e-mail goes to the registry mailbox (owner decision, 2026-10-06): the fixed recipients Send_Telemetry_Email.To are set to dgsregistry@nitda.gov.ng (the bulk-failure fallback was dgs@nitda.gov.ng). The configurable ones stay read from Flow Configuration (NOTIFY_OPS_TO, NOTIFY_REGISTRY_TO, NOTIFY_HELPDESK_TO), which fall back to the registry mailbox when blank.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_DGO_Scan_Notification_Email, Send_Telemetry_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message

body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Compose_Scan_Proof_Token`, `Get_Scan_Proof`, `Condition_Scan_Writeback`, `Compose_Scan_Writeback_Input`, `Compose_Scan_Writeback_Gate`, `Switch_Scan_Writeback_Gate`, `Set_variable_varStatusCode_Scan_Writeback_401`, `Append_to_array_variable_varErrors_Scan_Writeback_401`, `Compose_Value_Set_variable_varData_Scan_Writeback_401`, `Set_variable_varData_Scan_Writeback_401`, `Compose_Value_Set_variable_varStatusCode_Scan_Writeback_401`, `Set_variable_varStatusCode_Scan_Writeback_403`, `Append_to_array_variable_varErrors_Scan_Writeback_403`, `Compose_Value_Set_variable_varData_Scan_Writeback_403`, `Set_variable_varData_Scan_Writeback_403`, `Compose_Value_Set_variable_varStatusCode_Scan_Writeback_403`, `Set_variable_varStatusCode_Scan_Writeback_400`, `Append_to_array_variable_varErrors_Scan_Writeback_400`, `Compose_Value_Set_variable_varData_Scan_Writeback_400`, `Set_variable_varData_Scan_Writeback_400`, `Compose_Value_Set_variable_varStatusCode_Scan_Writeback_400`, `Get_Scan_Writeback_Submission`, `Condition_Scan_Writeback_Found`, `Update_Scan_Writeback_Registry`, `Create_Scan_Writeback_Timeline`, `Set_variable_varStatusCode_Scan_Writeback_200`, `Compose_Value_Set_variable_varData_Scan_Writeback_200`, `Set_variable_varData_Scan_Writeback_200`, `Compose_Value_Set_variable_varStatusCode_Scan_Writeback_200`, `Set_variable_varStatusCode_Scan_Writeback_404`, `Append_to_array_variable_varErrors_Scan_Writeback_404`, `Compose_Value_Set_variable_varData_Scan_Writeback_404`, `Set_variable_varData_Scan_Writeback_404`, `Compose_Value_Set_variable_varStatusCode_Scan_Writeback_404`, `Set_variable_varStatusCode_Scan_Writeback_500`, `Append_to_array_variable_varErrors_Scan_Writeback_500`, `Compose_Value_Set_variable_varData_Scan_Writeback_500`, `Set_variable_varData_Scan_Writeback_500`, `Compose_Value_Set_variable_varStatusCode_Scan_Writeback_500`, `Compose_Value_Set_variable_varData_Scan_RateLimited`, `Compose_Value_Set_variable_varData_Scan_BadRequest`, `Compose_Value_Set_variable_varData_Scan_Oversize`, `Compose_Value_Set_variable_varData_Scan_Duplicate`, `Compose_Scan_Cleared`, `Compose_Value_Set_variable_varData_Scan_Denied_403`, `Compose_Value_Set_variable_varData_Scan_Denied_401`, `Condition_Scan_Proceed`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`, `Compose_EntityKey_Record`, `Compose_Claim_Registry_Year`, `Get_Claim_Registry_Floor`, `Compose_Claim_Registry_Floor`, `Compose_Claim_Registry_Count`,

`Scope_Claim_Registry_1`, `Get_Counter_Registry_1`, `Condition_Counter_Registry_1_Exists`, `Compose_Next_Registry_1`, `Claim_Counter_Registry_1`, `Compose_Claimed_Registry_1`, `Create_Counter_Registry_1`, `Compose_Created_Registry_1`, `Scope_Claim_Registry_2`, `Delay_Claim_Registry_2`, `Get_Counter_Registry_2`, `Condition_Counter_Registry_2_Exists`, `Compose_Next_Registry_2`, `Claim_Counter_Registry_2`, `Compose_Claimed_Registry_2`, `Create_Counter_Registry_2`, `Compose_Created_Registry_2`, `Scope_Claim_Registry_3`, `Delay_Claim_Registry_3`, `Get_Counter_Registry_3`, `Condition_Counter_Registry_3_Exists`, `Compose_Next_Registry_3`, `Claim_Counter_Registry_3`, `Compose_Claimed_Registry_3`, `Create_Counter_Registry_3`, `Compose_Created_Registry_3`, `Scope_Claim_Registry_4`, `Delay_Claim_Registry_4`, `Get_Counter_Registry_4`, `Condition_Counter_Registry_4_Exists`, `Compose_Next_Registry_4`, `Claim_Counter_Registry_4`, `Compose_Claimed_Registry_4`, `Create_Counter_Registry_4`, `Compose_Created_Registry_4`, `Scope_Claim_Registry_5`, `Delay_Claim_Registry_5`, `Get_Counter_Registry_5`, `Condition_Counter_Registry_5_Exists`, `Compose_Next_Registry_5`, `Claim_Counter_Registry_5`, `Compose_Claimed_Registry_5`, `Create_Counter_Registry_5`, `Compose_Created_Registry_5`, `Scope_Claim_Registry_6`, `Delay_Claim_Registry_6`, `Get_Counter_Registry_6`, `Condition_Counter_Registry_6_Exists`, `Compose_Next_Registry_6`, `Claim_Counter_Registry_6`, `Compose_Claimed_Registry_6`, `Create_Counter_Registry_6`, `Compose_Created_Registry_6`, `Scope_Claim_Registry_7`, `Delay_Claim_Registry_7`, `Get_Counter_Registry_7`, `Condition_Counter_Registry_7_Exists`, `Compose_Next_Registry_7`, `Claim_Counter_Registry_7`, `Compose_Claimed_Registry_7`, `Create_Counter_Registry_7`, `Compose_Created_Registry_7`, `Scope_Claim_Registry_8`, `Delay_Claim_Registry_8`, `Get_Counter_Registry_8`, `Condition_Counter_Registry_8_Exists`, `Compose_Next_Registry_8`, `Claim_Counter_Registry_8`, `Compose_Claimed_Registry_8`, `Create_Counter_Registry_8`, `Compose_Created_Registry_8`, `Scope_Claim_Registry_9`, `Delay_Claim_Registry_9`, `Get_Counter_Registry_9`, `Condition_Counter_Registry_9_Exists`, `Compose_Next_Registry_9`, `Claim_Counter_Registry_9`, `Compose_Claimed_Registry_9`, `Create_Counter_Registry_9`, `Compose_Created_Registry_9`, `Scope_Claim_Registry_10`, `Delay_Claim_Registry_10`, `Get_Counter_Registry_10`, `Condition_Counter_Registry_10_Exists`, `Compose_Next_Registry_10`, `Claim_Counter_Registry_10`, `Compose_Claimed_Registry_10`, `Create_Counter_Registry_10`, `Compose_Created_Registry_10`, `Compose_Claim_Registry`, `Select_Claim_Registry_References`, `Condition_Scan_Claimed`, `Get_Scan_Registry_Mailbox`, `Compose_Scan_Sender`, `Compose_EntityKey_Scan_File`, `Create_Scan_Record`, `Create_Void_Registry`, `Condition_Scan_File_Stored`, `Scope_Update_Scan_File_1`, `Get_Row_Scan_File_1`, `Merge_Row_Scan_File_1`, `Compose_Rev_Scan_File_1`, `Scope_Update_Scan_File_2`, `Get_Row_Scan_File_2`, `Merge_Row_Scan_File_2`, `Compose_Rev_Scan_File_2`, `Scope_Update_Scan_File_3`, `Get_Row_Scan_File_3`, `Merge_Row_Scan_File_3`, `Compose_Rev_Scan_File_3`, `Compose_Updated_Scan_File`, `Compose_Value_Set_variable_varData_Scan_Deposited`, `Compose_Value_Set_variable_varData_Scan_AuditFailed`, `Compose_Value_Set_variable_varStatusCode_Scan_409`, `Compose_Value_Set_variable_varData_Scan_Lock`, `Compose_Value_Set_variable_varData_Scan_Catch`, `Create_Send_DGO_Scan_Notification_Email_Receipt`, `Compose_Send_DGO_Scan_Notification_Email_Outcome`, `Create_Send_Telemetry_Email_Receipt`, `Compose_Send_Telemetry_Email_Outcome`.

Steps removed: `Compose_Scan_Year`, `Compose_Scan_LockToken`, `Get_Scan_Sequence_Counter`, `Condition_Scan_Sequence_Exists`, `Claim_Scan_Sequence_Counter`, `Create_Scan_Sequence_Counter`, `Get_Scan_Sequence_Confirm`, `Condition_Scan_Lock_Held`, `Compose_Scan_Next_Sequence`, `Update_Scan_Sequence_Counter`.

Test after pasting. Deposit one scan from Folio. The screen shows a NITDA-YYYY-NNNNN reference (it showed none before). In Portal Registry the new row has Channel scan, SenderEmail = the registry mailbox, EntityKey, IdState ACTIVE and Rev 1; in NITDA_Central_Registry the file is named *<8 hex>* and its EntityKey begins with those 8 hex; DGO_AuditLog has the deposit event with RootRef = the reference. Then T2: a portal submission and a scan in the same second get consecutive numbers.

Phase 3 tenant tests (§14.3)

Run once every Phase 3 flow is pasted (§13: Phase 3 exits on flow lint, which `npm run test:idphase3` is, and T1–T6):

#	Test	Flow	Pass condition
T1	20 concurrent portal submissions	SUBMISSION	20 distinct, consecutive NITDA-YYYY-NNNNN; no gaps; the counter equals the last
T2	A portal submission and a scan intake in the same second	SUBMISSION, SCAN_INTAKE	Consecutive numbers from the one counter. Waits on SCAN_INTAKE (an internal flow, Phase 4 in this repo's sequence).
T3	The same content twice within 24 hours	SUBMISSION	The second call returns the first reference; nothing new is issued
T4	The client sends `requestId`, `ref` and `localId`	SUBMISSION, SUPPORT	Ignored; logged as ClientCorrelationId only
T5	A support case naming a reference	SUPPORT	`^CASE-[A-HJ-NP-Z2-9]{6}$`; an ABOUT link when the record has an Entity Key
T6	STATUS with NITDA-2026-217 (legacy) and with NITDA-2026-00217	STATUS	Each resolves to its own record, or not found; never to the other

Annex B · Workspace and scheduled packages

IDENTIFICATION_SYSTEM.md §13, Phase 4. Each file is the flow's root scope as the tenant capture holds it (`docs/folio-Master-Flows-and-Endpoints/captures/`), with the edits below applied. `tests/identification-phase4.test.mjs` runs every file in the strict simulator and checks the designer's limits (nesting depth 8, no variable declared inside a pasted scope).

Before pasting

Phases 1 to 3 as recorded in identification-phase1, identification-phase2 and identification-phase3. The first TASK claim of a year creates the counter row `TASK-<year>` in Portal Sequence Counters; nothing has to be created by hand.

Flow Configuration for the new lists

FETCH_ALL (and DYNAMIC_ACTIONS) address the lists Phase 1 created through Flow Configuration. The provisioning template shipped these rows as "(set after the list is created)". Each value below is the list as the Phase 1 list check found it (p1-1-list-check-2026-10-04-after-finish.json). Where a row still holds the placeholder, set it to the value shown. A row left unset only empties that collection.

Title	ConfigValue
DATASET_FLOW_REGISTRY_SITE_URL	https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE
DATASET_FLOW_REGISTRY_LIST_ID	d9a6eef7-cb58-497b-a478-9f526d4a9d8b
DATASET_FLOW_DEPENDENCIES_SITE_URL	https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE
DATASET_FLOW_DEPENDENCIES_LIST_ID	f2cd5180-f1a0-4433-be2d-bc8ddfa0291c
DATASET_RECORD_LINKS_SITE_URL	https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE
DATASET_RECORD_LINKS_LIST_ID	c3875803-8a9c-42d6-86b9-85e3f0ca5eef
DATASET_IDENTITY_MAP_SITE_URL	https://nitdanigeria.sharepoint.com/sites/DGO_ECM_GOVERNANCE
DATASET_IDENTITY_MAP_LIST_ID	b25086d4-15da-493a-b45e-a8d068dc6220
DATASET_BRIEFS_SITE_URL	https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING
DATASET_BRIEFS_LIST_ID	e73a701a-fbcd-4d1d-b800-9fe26d12b2b6

Title	ConfigValue
DATASET_MEETINGS_SITE_URL	https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING
DATASET_MEETINGS_LIST_ID	6987151e-f4a7-4cff-9d4b-57f92383ef0b
DATASET_PROJECTS_SITE_URL	https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING
DATASET_PROJECTS_LIST_ID	38cf8739-4a63-48fb-9351-7373b797db7c
DATASET_APPROVALS_SITE_URL	https://nitdanigeria.sharepoint.com/sites/NITDADGO-EAAACTIVITYTRACKING
DATASET_APPROVALS_LIST_ID	5bcb98f3-3a86-408b-aa91-a75165b8246e
DATASET_REGISTRY_FILES_SITE_URL	https://nitdanigeria.sharepoint.com/sites/NEDMS
DATASET_REGISTRY_FILES_LIST_ID	98e85c69-d72a-4e8b-b1b2-1809fec74786
DATASET_FILE_MOVEMENTS_SITE_URL	https://nitdanigeria.sharepoint.com/sites/NEDMS
DATASET_FILE_MOVEMENTS_LIST_ID	0c3d7dec-6a1a-49d8-b3c2-b21c7fd85521

How to apply

All at once: `npm run flows:update -- apply` writes every package below into its flow, backing each flow up first and reading it back (FLOWS_UPDATE.md). Or by hand, one flow at a time:

For each flow: open it, delete the root scope named below, select the step above where it was, choose **Paste an action**, paste the file, and save. Keep every **Initialize variable** step above it: these scopes set the flow's own variables. If a step shows *Invalid connection*, open it and choose the existing connection once. Then run the flow's test.

Flow	Flow ID	Root scope	Steps	File	In the tenant
Single_Task_Assignment	3caf9458-8d95-4740-a01b-6803dad97d34	Scope_Global	191	SINGLE_ASSIGNMENT__Single_Task_Assignment__3caf9458.paste.json	In the tenant (2026-10-19)

Flow	Flow ID	Root scope	Steps	File	In the tenant
BULK_OPS_DATA_RETRIEVAL_HTTP (Folio's BULK_ASSIGNMENT)	`5aab9e61-ab9f-4af1-9f2b-eaba23a1bf06`	`Scope_Global`	177	`BULK_ASSIGNMENT__BULK_OPS_DATA_RETRIEVAL_HTTP__5aab9e61.paste.json`	In the tenant (2026-10-19)
IP_FETCH_ALL_BOOTSTRAP (Folio's FETCH_ALL)	`b79e6707-df4d-4ae7-8a65-738aa45e9a73`	`Scope`	96	`FETCH_ALL__IP_FETCH_ALL_BOOTSTRAP__b79e6707.paste.json`	In the tenant (2026-10-19)
IP_Dynamic_Global_Actions_Endpoint (Folio's DYNAMIC_ACTIONS)	`31c0aaf0-ddbb-478c-9a98-bc013265c4c7`	`Scope_Dynamic_Flow`	480	`DYNAMIC_ACTIONS__IP_Dynamic_Global_Actions_Endpoint__31c0aaf0.paste.json`	In the tenant (2026-10-19)
IP_User_Admin_Endpoint (Folio's USER_ADMIN)	`e3e5c825-55ba-447e-88c1-96b7e89fd013`	`Scope_Dynamic_Flow`	97	`USER_ADMIN__IP_User_Admin_Endpoint__e3e5c825.paste.json`	In the tenant (2026-10-19)
IP_Activity_Lifecycle_Endpoint (Folio's ACTIVITY_LIFECYCLE)	`b4db02d8-02a0-4bc9-b9d3-06e2aae2448c`	`Scope_Dynamic_Flow`	90	`ACTIVITY_LIFECYCLE__IP_Activity_Lifecycle_Endpoint__b4db02d8.paste.json`	In the tenant (2026-10-19)

Flow	Flow ID	Root scope	Steps	File	In the tenant
IP_Create__Task__for_received_Email_Endpoint (Folio's EMAIL_RELATED_TASK)	`ab1a3555-7fce-4bfe-a193-b8b84654b0a4`	`Scope_Global`	179	`EMAIL_RELATED_TASK__IP_Create_Task_for_received_Email_Endpoint__ab1a3555.paste.json`	In the tenant (2026-10-19)
IP_OTP_VERIFY (Folio's OTP_VERIFY)	`4dbe6dde-37d5-47b4-ba32-d4ae5c9f8b61`	`Scope_Global`	51	`OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde.paste.json`	In the tenant (2026-10-18)
IP_OTP_GENERATE (Folio's OTP_GENERATE)	`e5632336-641f-4b9b-bd93-99185b5eb975`	`Scope_Global`	29	`OTP_GENERATE__IP_OTP_GENERATE__e5632336.paste.json`	In the tenant (2026-10-19)
IP_Reference_Data_Endpoint (Folio's REFERENCE_DATA)	`9a4dea8e-15f2-42b6-904d-3655056fa3c2`	`Scope_Global`	56	`REFERENCE_DATA__IP_Reference_Data_Endpoint__9a4dea8e.paste.json`	In the tenant (2026-10-18)
IP_Get_Docs_Endpoint (Folio's GET_DOCS)	`28f6e62a-fea8-437c-a61a-47aec706c4e2`	`Scope_Global`	52	`GET_DOCS__IP_Get_Docs_Endpoint__28f6e62a.paste.json`	In the tenant (2026-10-18)

Flow	Flow ID	Root scope	Steps	File	In the tenant
IP_Retrieve_Email_Attachment_Endpoint (Folio's FETCH_EMAIL_ATTACHMENTS)	`5bc87cf3-7a99-4259-b6a2-315fbc794d9b`	`Scope_Email_Attachment_Endpoint_REBUILT`	38	`FETCH_EMAIL_ATTACHMENTS__IP_Retrieve_Email_Attachment_Endpoint__5bc87cf3.paste.json`	In the tenant (2026-10-18)
IP_SEND_EMAIL (Folio's EMAIL)	`df59a69f-92ca-4554-ae68-6b464aa73ef7`	`Scope_Send_Email_Flow`	73	`EMAIL__IP_SEND_EMAIL__df59a69f.paste.json`	In the tenant (2026-10-19)
AI_Document_Processing (Folio's AI)	`fefa43c6-fb9d-4337-8b65-7fd19fa09b6d`	`Scope_AI_Caller_Gate`	12	`AI_DOC_ANALYSIS__AI_Document_Processing__fefa43c6.paste.json`	In the tenant (2026-10-18)
Folio_Incoming_Correspondence_Scan_Processing_Flow (Folio's SCAN_PROCESSING)	`197527c8-e76c-4f7d-81b8-3cb5c88b1c6c`	`Scope_Scan_Processing`	217	`NOKEY__Folio_Incoming_Correspondence_Scan_Processing_Flow__197527c8.paste.json`	In the tenant (2026-10-19)
AUTO_SCHEDULED_SWEEP (Folio's SWEEP)	`8c19b45c-4174-4d2f-9429-ffd08dd1a5ae`	`Scope_ScheduledSweep_Flow`	103	`NOKEY__AUTO_SCHEDULED_SWEEP__8c19b45c.paste.json`	In the tenant (2026-10-19)

Flow	Flow ID	Root scope	Steps	File	In the tenant
Folio_Failure_Watch (Folio's FAILURE_WATCH)	`c38bfeaa-4c88-47a5-a9e2-f5d0e610efb6`	`Scope_Failure_Watch`	34	`NOKEY__Folio_Failure_Watch__c38bfeaa.paste.json`	In the tenant (2026-10-19)

Caller identity (G-04)

Every internal endpoint key resolves its caller the same way (SINGLE_ASSIGNMENT, BULK_ASSIGNMENT, FETCH_ALL, DYNAMIC_ACTIONS and SUBSIDIARY_ACTIONS, EMAIL_RELATED_TASK, REFERENCE_DATA, GET_DOCS and FETCH_ACTIVITIES, FETCH_EMAIL_ATTACHMENTS, EMAIL and the three AI keys), through one Flow Configuration switch, `IDENTITY_PROOF_REQUIRED`:

Switch	Caller	Refused
not `true` (no row, or any other value)	the sign-in proof's address when a valid proof is sent, else the address the request names (`userEmail`; Bulk also `requestedBy`)	as before these packages: FETCH_ALL, DYNAMIC_ACTIONS and EMAIL by their own directory and role gate; the others not at all (REFERENCE_DATA and GET_DOCS had a gate whose decision was hard-coded to pass)
`true`	only the sign-in proof's address: the bearer token Folio sends, found in the proof ledger unconsumed, Purpose internal-session and unexpired	no valid proof, or not an active directory user: 401. A role without the endpoint's route (single-assignment, bulk-assignment; `"*"` and `bulk:assign` also pass): 403

This is the enforcement of G-04, carried by the identification packages so that applying it is setting the switch, not pasting other flows over these. The proof is minted by IP_OTP_VERIFY (`OTP_VERIFY__IP_OTP_VERIFY__4dbe6dde.paste.json`, below). Set the switch only after that package is pasted and Folio users can sign in: before then no client holds a valid proof and every call would be refused. The DGO_* replacement packages that carried an earlier form of this are retired (2026-10-06, registers/flow-retirement.json).

REFERENCE_DATA and GET_DOCS do not yet enforce a role's route or permission, only the caller: which roles hold them is not recorded in this repository. REFERENCE_DATA serves the full directory (Role, Persona, Status, AccessScope, PilotCohort) only to a holder of `user:view` or `"*"`, and to anyone else on the caller's own row.

The directory row's `Role` is a Choice column (docs/reference/sharepoint-provisioning-spec.json). FETCH_ALL and DYNAMIC_ACTIONS read it as text, which never matched a `RoleId`; every package here reads its value.

One bulk assignment endpoint

Folio calls one bulk endpoint, `BULK_ASSIGNMENT` (modules/bulk-assignment.js). Its URL is the trigger of BULK_OPS_DATA_RETRIEVAL_HTTP (`5aab9e61`), which only read documents, so a bulk assignment created no task. That flow now holds the bulk assignment scope below and is the one bulk endpoint kept. `BULK_ASSIGNMENT_DIRECT` (IP_Bulk_Assign_Endpoint, `0d1e8df5`, reassignment only) is called by nothing in Folio and is retired: turn that flow off, and leave its URL out of the endpoint configuration. The legacy Deployed Bulk Task Assignment_Create Task (`122bb363`) is not wired to Folio and stays off.

Changes

Single_Task_Assignment

- S8 / ID-19: every run mints its Request ID. varRequestId was declared with no value and never set, so every reply and error record carried an empty requestId. The root scope now sets it to guid() first.
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. With the switch true, a caller who is not an active directory user is refused with 401 and one whose role does not allow single-assignment (or bulk:assign) with 403, through the flow's own validation-failure path, before any task is claimed or written. The task's AssignedBy is that caller.
- ID-19: the reply's requestId is always the flow's own. It took the client's requestId whenever the request carried one.
- S3 / §8.2: each new task gets a Task Reference TASK-YYYY-NNNNN, claimed by the generation kit (10 flat attempts, IF-MATCH, counter row TASK- created by the first claim of the year), addressed through Flow Configuration (DATASET_SEQUENCE_COUNTERS_*). If no number can be claimed the task is not created and the run takes the existing Task Processing catch, which records the error.
- §10.2 TASK_QUEUE: the task row carries its Entity Key, TaskRef, ParentKey and RootKey REC: <the document's record key> (DGO DIGITAL OPS rows carry the master record's key; empty for a row the Phase 5 backfill has not reached), RootRef (the record's ReferenceId), IdState ACTIVE, Rev 1, the Request ID as CorrelationId and CreatedByRequest, and ClientRequestHash when the request carries one (B3). The task type and grouping, which a second Update item wrote straight after the create, are written by the create itself, so that update is removed (it is a governed row: B9).
- S3: new tasks have no composite reference. Compose_ReferenceID, which the e-mails, the acknowledgement link's ref= text and the task/document pairs show, is now the Task Reference. The composite ---- was never stored, so nothing that resolves a task changes; the acknowledgement link finds the task by its item= value, as before.
- Review C-1 (2026-10-02): assigning a document no longer erases its AI extraction. Update_item_Doc wrote BodyExtract and TextExtraxt as empty text; both keys are removed,

and Update item leaves fields it is not given unchanged.

- Review C-2 (2026-10-02): Compose_Config_BaseURL held a Markdown link, https://taskack.page.gd, so the acknowledgement link built from it was broken. It is now the address itself.
- Tenant defect 2026-10-10 (TASK-2026-00002, run 08584100110433772866731334335CU24): the e-mail greeted "Dear ,". Compose_AssignedToTitle_Value took coalesce() of its candidates, and coalesce() skips only nulls: the department match came back as empty text and won. It now takes the first candidate that is not empty, ending with "Officer".
- Tenant defect 2026-10-10: Handling Status and Response Priority both read "Standard Official Review" on every normal task, because both fell back to that text. Response Priority now states the response window it describes: "Priority Review Window" for critical, urgent or high, otherwise "Standard Response Window". Handling Status is unchanged.
- Tenant defect 2026-10-10: the Support Access link sent ref=TASK- (TASK-16167) while the e-mail itself shows the Task Reference (TASK-2026-00002). The link now carries the Task Reference, Compose_ReferenceID, and keeps item= for the lookup.
- Deep links: task e-mail links start at https://folio-platform.page.gd (the Workspace's primary host, owner 2026-10-08), not https://taskack.page.gd.
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_varData_Validation_Failure, Set_variable_varRequestId, Set_variable_varStatusCode.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_an_email_(V2)*Main_5): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose*_Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.
- ITEM-77 (owner decision, 2026-10-06): every Office 365 step signs in as the registry mailbox, dgsregistry@nitda.gov.ng, connection c0b9e7a5 (Send_an_email_(V2)_Main_5 used another connection).

Steps added: `Compose_Value_Set_variable_varStatusCode`, `Compose_Value_Set_variable_varData_Validation_Failure`, `Create_Send_an_email_(V2)_Main_5_Receipt`, `Compose_Send_an_email_(V2)_Main_5_Outcome`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`, `Compose_EntityKey_Task`, `Compose_Claim_Task_Year`, `Compose_Claim_Task_Count`, `Scope_Claim_Task_1`, `Get_Counter_Task_1`, `Condition_Counter_Task_1_Exists`, `Compose_Next_Task_1`, `Claim_Counter_Task_1`, `Compose_Claimed_Task_1`, `Create_Counter_Task_1`, `Compose_Created_Task_1`, `Scope_Claim_Task_2`, `Delay_Claim_Task_2`, `Get_Counter_Task_2`, `Condition_Counter_Task_2_Exists`, `Compose_Next_Task_2`, `Claim_Counter_Task_2`, `Compose_Claimed_Task_2`, `Create_Counter_Task_2`, `Compose_Created_Task_2`, `Scope_Claim_Task_3`, `Delay_Claim_Task_3`, `Get_Counter_Task_3`, `Condition_Counter_Task_3_Exists`, `Compose_Next_Task_3`, `Claim_Counter_Task_3`, `Compose_Claimed_Task_3`, `Create_Counter_Task_3`, `Compose_Created_Task_3`, `Scope_Claim_Task_4`,

`Delay_Claim_Task_4`, `Get_Counter_Task_4`, `Condition_Counter_Task_4_Exists`, `Compose_Next_Task_4`, `Claim_Counter_Task_4`, `Compose_Claimed_Task_4`, `Create_Counter_Task_4`, `Compose_Created_Task_4`, `Scope_Claim_Task_5`, `Delay_Claim_Task_5`, `Get_Counter_Task_5`, `Condition_Counter_Task_5_Exists`, `Compose_Next_Task_5`, `Claim_Counter_Task_5`, `Compose_Claimed_Task_5`, `Create_Counter_Task_5`, `Compose_Created_Task_5`, `Scope_Claim_Task_6`, `Delay_Claim_Task_6`, `Get_Counter_Task_6`, `Condition_Counter_Task_6_Exists`, `Compose_Next_Task_6`, `Claim_Counter_Task_6`, `Compose_Claimed_Task_6`, `Create_Counter_Task_6`, `Compose_Created_Task_6`, `Scope_Claim_Task_7`, `Delay_Claim_Task_7`, `Get_Counter_Task_7`, `Condition_Counter_Task_7_Exists`, `Compose_Next_Task_7`, `Claim_Counter_Task_7`, `Compose_Claimed_Task_7`, `Create_Counter_Task_7`, `Compose_Created_Task_7`, `Scope_Claim_Task_8`, `Delay_Claim_Task_8`, `Get_Counter_Task_8`, `Condition_Counter_Task_8_Exists`, `Compose_Next_Task_8`, `Claim_Counter_Task_8`, `Compose_Claimed_Task_8`, `Create_Counter_Task_8`, `Compose_Created_Task_8`, `Scope_Claim_Task_9`, `Delay_Claim_Task_9`, `Get_Counter_Task_9`, `Condition_Counter_Task_9_Exists`, `Compose_Next_Task_9`, `Claim_Counter_Task_9`, `Compose_Claimed_Task_9`, `Create_Counter_Task_9`, `Compose_Created_Task_9`, `Scope_Claim_Task_10`, `Delay_Claim_Task_10`, `Get_Counter_Task_10`, `Condition_Counter_Task_10_Exists`, `Compose_Next_Task_10`, `Claim_Counter_Task_10`, `Compose_Claimed_Task_10`, `Create_Counter_Task_10`, `Compose_Created_Task_10`, `Compose_Claim_Task`, `Select_Claim_Task_References`, `Compose_Task_Claim_Check`, `Compose_Task_Ref`, `Set_variable_varRequestId`, `Get_Assign_Gate_Proof_Mode`, `Compose_Assign_Gate_Proof_Token`, `Get_Assign_Gate_Proof`, `Compose_Assign_Gate_Identity`, `Compose_Assign_Gate_Caller`, `Get_Assign_Gate_Directory_User`, `Compose_Assign_Gate_Role`, `Get_Assign_Gate_Role_Permissions`, `Compose_Assign_Gate_Auth`, `Compose_Value_Set_variable_varRequestId`.

Steps removed: `Update_item_Task`.

Test after pasting. Assign one document from Folio's Assignment Desk. The officer's e-mail shows a TASK-YYYY-NNNNN reference; in Global Tracking Queue the new row has TaskRef = that reference, EntityKey, IdState ACTIVE, Rev 1, TaskTypeChoice Standard Task, and ParentKey REC: when the document's DGO DIGITAL OPS row has an EntityKey; the document row keeps its BodyExtract and TextExtraxt; the reply carries a requestId.

BULK_ASSIGNMENT

- Each document goes through the task steps of the Single_Task_Assignment package above: the task row in Global Tracking Queue with its identity columns, the document row in DGO DIGITAL OPS (Processed, its AI extraction kept: C-1), the assignee's title from Departments, and one notification to the assignee with the task's acknowledgement link. A bulk task and a single task are the same record.
- T7 / §8.2: the Task References for the whole batch are claimed once, by the generation kit, before any task is created: N consecutive TASK-YYYY-NNNNN numbers from the TASK-counter. Each document is paired with its number before the loop, so a document keeps its number whatever happens to the others. If no numbers can be claimed the reply is 503 RETRY_LATER and nothing is created.

- Folio's payload is read as it sends it (core/assignment-payload.js, dgo-bulk-assignment-payload/v2: ids, category, categoryCode, subcategory, subcategoryCode, assignedTo, assignedToDsu, supportingAssignee, supportingDsu, ccRecipients, priority, startDate, ack, due, instruction, requestedBy). Compose_Bulk_Task holds those values under the names the task steps read; per document, the activity and document IDs are the document's own and the title and attachment link come from its DGO DIGITAL OPS row.
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above), from the sign-in proof or, while IDENTITY_PROOF_REQUIRED is not true, from userEmail or requestedBy. With the switch true, a caller who is not an active directory user is refused with 401 and one whose role does not allow bulk-assignment (or bulk:assign) with 403, before any number is claimed. The batch's requester (AssignedBy, the failure e-mail) is that caller.
- The request is refused with 400 unless it names 1 to 50 documents (Folio's own limit, config/app.config.js maxBulkAssign), an assignee e-mail address and a category.
- The reply (202 with the claimed references, or 400, 503, 500) is sent before any task is created, with Access-Control-Allow-Origin from Flow Configuration's ALLOWED_ORIGIN rows, as the flow's data-retrieval reply did. The documents are then assigned one after another. A document whose task or update fails is recorded with its reference in varErrors and the rest go on; if any failed, the requester (requestedBy, else dgs@nitda.gov.ng) gets one e-mail listing them.
- S8 / ID-19: the run sets its own Request ID (varRequestId = guid()) first; the reply, the task rows and the failure e-mail carry it. Nothing the client sends is adopted.
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_varData_Bulk_Accepted, Set_variable_varData_Bulk_Empty, Set_variable_varData_Bulk_Invalid, Set_variable_varData_Bulk_Retry.
- Administrative e-mail goes to the registry mailbox (owner decision, 2026-10-06): the fixed recipients Send_Bulk_Failure_Email.To are set to dgsregistry@nitda.gov.ng (the bulk-failure fallback was dgs@nitda.gov.ng). The configurable ones stay read from Flow Configuration (NOTIFY_OPS_TO, NOTIFY_REGISTRY_TO, NOTIFY_HELPDESK_TO), which fall back to the registry mailbox when blank.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Bulk_Failure_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.
- ITEM-77 (owner decision, 2026-10-06): every Office 365 step signs in as the registry mailbox, dgsregistry@nitda.gov.ng, connection c0b9e7a5 (Send_an_email_(V2)_Main_5, Send_Bulk_Failure_Email used another connection).

The scope is new: it replaces the flow's Scope_Global as a whole (177 steps).

Test after pasting. Assign three documents to one officer from Folio's Bulk Assignment. The reply is 202 with three consecutive TASK-YYYY-NNNNN references; in Global Tracking Queue three new rows carry those TaskRefs, each with its own EntityKey, IdState ACTIVE and Rev 1; each document

row shows AssignmentStatus Processed and keeps its BodyExtract; the officer receives three e-mails, one per task; Portal Sequence Counters' TASK- row advanced by three in one claim. Then send one document ID that does not exist with two that do: two tasks are created and the requester receives one e-mail naming the missing one.

FETCH_ALL

- S8 / ID-19: the run sets its own Request ID (varRequestId = guid()) first. The flow declared it as the client's requestId when one was sent.
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. With the switch true and no valid proof, the flow's own unknown-caller branch answers 401. The role is read from the directory row's Role choice, which the flow read as text and so never matched a RoleId.
- §10.2: every record the Workspace reads carries its identity. Documents: EntityKey, ReferenceId, IdState. Tasks: EntityKey, TaskRef, ParentKey, RootKey, RootRef, IdState, Rev. Comments (minutes): EntityKey, ParentKey, RootKey, RootRef, AuthorKey. The fields already returned are unchanged.
- §13 Phase 4: FETCH_ALL also returns register, briefs, meetings, projects, approvals, files: the latest 500 rows of Portal Registry, DGO_Briefs, DGO_Meetings, DGO_Projects, DGO_Approvals and DGO_RegistryFiles, each with its Entity Key and reference, addressed through Flow Configuration (DATASET__*). A collection whose list cannot be read comes back empty and the rest of the reply is unaffected. approvals, which was always empty, now holds DGO_Approvals rows in the shape Folio's approvals view reads.
- Directory and reference data as Folio reads them, ported from the retired DGO_FETCH_ALL design (registers/flow-retirement.json): the directory is read in pages to 5,000 rows (it holds 852; one GetItems page is 500), Role, Persona and Status are served as text rather than Choice objects, and Departments and Categories, imported from spreadsheets so GetItems returns field_1…field_N, are read by those internal names. The keys Folio reads are unchanged.
- ITEM-90 (owner approval 2026-10-08): every active directory user is admitted and receives the slice of data their role allows; until now only roles carrying "*" or user:view were admitted (systemAdmin and userAdmin: the operator and 822 active viewers got 403), and whoever was admitted received everything. Tiers, from the directory row: all (systemAdmin, userAdmin, executive, operator, which is the registry role, or the admin, executive or registry persona) receives what it did; every other caller receives the tasks assigned to them, read from Global Tracking Queue by the indexed AssignedTo so older assignments are included, the documents those tasks name (read by ID, up to 60), and those documents' comments. A director also receives approvals; their directorate's tasks are not read, because Global Tracking Queue holds its 20 indexed columns and GDSUROUT is not one of them (follow-up). The directory goes to every caller, with e-mail addresses only for the all tier and on the caller's own row. Portal submissions, briefs, meetings, projects and registry files go to the all tier only. Tasks now carry RoutedDsu (GDSUROUT).
- Owner decision 2026-10-18: the Workspace's System Health shows failed notices. FETCH_ALL returns failureHealth, the JSON Folio_Failure_Watch writes to Flow Configuration

NOTIFY_FAILURE_HEALTH (the open count per group, the broken-connection flag and the time of the check), to the all tier only; null when the row is absent or unreadable. One row of a few hundred characters: the reply grows by that and nothing else.

- Tenant defect (8 October 2026): the newest 500 tasks are read by ID, not Modified. Global Tracking Queue holds 16,071 rows and already carries the 20 indexed columns SharePoint allows, none of them Modified, so "Modified desc" exceeded the list view threshold: Get_All_Tasks failed, every read after it was skipped, and the flow answered 200 with null collections, so the Workspace loaded empty. ID is always indexed.
- ITEM-74 (owner decision 2026-10-10): FETCH_ALL carries the registry inbox. For the all tier (the registry, admin and executive roles) it reads the newest 50 messages of the registry mailbox's Inbox (Office 365 Get emails (V3), as Fetch_Emails_POST does, no attachments) and returns id, subject, sender, received time, preview, importance, conversation, internetMessageId and whether it has attachments, so Lookup can raise a task from an e-mail (EMAIL_RELATED_TASK). Other tiers receive none. A failed read leaves the list empty; the rest of the reply is unchanged.
- ITEM-52: the Response_FetchAll run record carries the reply's size instead of the reply, and the read endpoint no longer e-mails its run record (owner decision 2026-10-06; tenant evidence 2026-10-10).
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_varData_FetchAll_Catch, Set_variable_varData_FetchAll_Denied_401, Set_variable_varData_FetchAll_Denied_403, Set_variable_varData_FetchAll_Success, Set_variable_varRequestId.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.

Steps added: `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Get_FetchAll_Register`, `Select_FetchAll_Register`, `Get_FetchAll_Briefs`, `Select_FetchAll_Briefs`, `Get_FetchAll_Meetings`, `Select_FetchAll_Meetings`, `Get_FetchAll_Projects`, `Select_FetchAll_Projects`, `Get_FetchAll_Approvals`, `Select_FetchAll_Approvals`, `Get_FetchAll_Files`, `Select_FetchAll_Files`, `Compose_FetchAll_Scope`, `Condition_FetchAll_Own`, `Get_FetchAll_Own_Tasks`, `Select_FetchAll_Own_Tasks`, `Get_FetchAll_Unit_Map`, `Compose_FetchAll_Units`, `Get_FetchAll_Directorate_Tasks`, `Select_FetchAll_Directorate_Tasks`, `Compose_FetchAll_Scope_Tasks`, `Select_FetchAll_Own_Doc_Ids`, `Get_FetchAll_Own_Docs`, `Select_FetchAll_Own_Docs`, `Get_FetchAll_Own_Comments`, `Select_FetchAll_Own_Comments`, `Select_FetchAll_Scoped_Users`, `Get_FetchAll_Failure_Health`, `Condition_FetchAll_Read_Emails`, `Get_FetchAll_Registry_Emails`, `Select_FetchAll_Emails`, `Compose_Value_Set_variable_varData_FetchAll_Success`, `Compose_Value_Set_variable_varData_FetchAll_Denied_403`, `Compose_Value_Set_variable_varData_FetchAll_Denied_401`, `Get_FetchAll_Gate_Proof_Mode`, `Compose_FetchAll_Gate_Proof_Token`, `Get_FetchAll_Gate_Proof`, `Compose_FetchAll_Gate_Identity`, `Compose_FetchAll_Gate_Caller`, `Compose_Value_Set_variable_varData_FetchAll_Catch`, `Set_variable_varRequestId`, `Compose_Value_Set_variable_varRequestId`.

Steps removed: `Compose_Telemetry_Attachments`, `Send_Telemetry_Email`.

Test after pasting. Open Folio. The Workspace loads as before; in the browser's network tab the FETCH_ALL reply carries a requestId of its own and data.register, briefs, meetings, projects, approvals and files arrays (empty where the list has no rows yet); data.docs rows carry EntityKey and ReferenceId, data.tasks rows TaskRef and EntityKey.

DYNAMIC_ACTIONS

- S8 / ID-19: the Request ID is the flow's own guid(). It was the client's X-Correlation-ID or requestId whenever one was sent; that value is now kept only as clientCorrelationId (sanitised, 64 characters).
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. With the switch true and no valid proof, the gate answers 401. The role is read from the directory row's Role choice, which the flow read as text and so never matched a RoleId.
- The request may now carry, at its top level or under payload, the fields the identity operations read: entityType, entityKey, target, survivor, fromKey, toKey, linkType, title, text, localKey, parentKey, reason, toOrgKey, movementAction, subject, senderEmail, channel, locatorKind, locatorContainer, locatorValue, legacyNumber, decision and clientRequestHash.
- The gate serves the new operations (register, upsert-record, open-file, move-file, link, void, reparent, merge; underscores are read as hyphens, so upsert_record is upsert-record). upsert-record leaves the recognised-but-unserved set. Routes the caller's role must allow: register and link need correspondence; upsert-record needs briefs, meetings, projects, approvals or orchestrator (minutes) by entity type; open-file, move-file, void and reparent need registry; merge needs registry-merge (Registry Manager only, §9: add "registry-merge" to that role's AllowedRoutesJson; "*" also passes).
- Each new operation is checked before anything is written; a request that fails is refused with 400 and the reason. register: a subject. upsert-record: entity type MIN, BRF, MTG, PRJ, APR or RMD (a reminder: assignedTo and dueAt, written once); a title (or, for a minute, its text and its task as parentKey TSK:) unless entityKey names the row to change. open-file: a title. move-file: the file as target and toOrgKey. link: fromKey and toKey as Qualified Keys and one of the 11 link types. void: a target and a reason. reparent: a target ATT, TSK, MIN or APR Qualified Key and the new parentKey. merge: the absorbed record as target and the survivor, two different REC Qualified Keys. A target is a Qualified Key TYPE: or spo:/ (B10); a bare item ID, RefIDD or title is refused.
- T11 / B3: a replayed request (the same idempotency key; Folio sends its clientRequestHash as that key) returns the first result unchanged. Every new operation stores its result on its audit event, and the replay answer is that result; older events, which hold none, keep the replay answer they had.
- ITEM-81 (from the hardened scope): an idempotency key already used for a different operation answers 409 IDEMPOTENCY_CONFLICT and nothing is replayed or written; only the same request gets the first result back.

- register (§8.2, S7): registers a record received outside the portal (Folio, e-mail, library). A request whose clientRequestHash Portal Registry already holds returns that record. Otherwise it claims the next NITDA-YYYY-NNNNN (the kit claim, floored at the register's highest number) and writes the Portal Registry row: Title and ReferenceId, Subject, Channel (one of the catalogue's channel codes, else internal), CorrespondenceType Incoming, SenderEmail, Status received, EntityKey, IdState ACTIVE, Rev 1, Direction In, CorrelationId, CreatedByRequest, ClientRequestHash and, when sent, SourceLocator. A locator (locatorKind EXO_INTERNET_MESSAGE_ID, SPO_ITEM, ODB_ITEM or PAPER_FILE, with locatorContainer and locatorValue; for an e-mail, the registry mailbox and the internetMessageId, T8) is written to DGO_IdentityMap as a LOCATOR entry: System from the catalogue, MatchKey LOCATOR||| (§10.2), not written again when that key exists. If the row cannot be written a VOID row accounts for the number (Q2) and the reply is 500; if no number can be claimed, 503. The audit event REGISTERED carries REC:.
- upsert_record (§7, §13): creates or changes a minute, brief, meeting, project or approval. With entityKey it changes that row's Title (a minute's Description, an approval's Decision) by the kit's Guarded Update (Rev + 1); without, it creates the row with a new Entity Key, IdState ACTIVE, Rev 1, CreatedByRequest and ClientRequestHash (CorrelationId where the list has it), after returning the existing row if its clientRequestHash is already there. A minute (Task_Comments) is created under its task: ParentKey TSK:, RootKey and RootRef from the task row, RefIDD the task's document, Description the text, AuthorKey USR:; an unknown task is 404. An approval takes Decision (Pending unless sent) and, when parentKey is given, ParentKey and RootKey. When Folio sends its own local key (localKey), DGO_IdentityMap gets a FOLIO_LOCAL_KEY alias, System runtime, MatchKey ALIAS|FOLIO_LOCAL_KEY|| (§12.3 4a).
- open_file (§8.2, T12): opens a registry file. A request whose clientRequestHash DGO_RegistryFiles already holds returns that file. Otherwise it claims the next FILE-YYYY-NNNNN (the kit claim, counter FILE-) and writes the file: Title, RegistryFileRef, CurrentHolder (toOrgKey, else the caller), EntityKey, IdState ACTIVE, Rev 1, CorrelationId, CreatedByRequest, ClientRequestHash; then the OPENED movement in DGO_FileMovements; and, when Folio sends the file's old DGO/YYYY/NNNNNN number (legacyNumber), a LEGACY_REGISTRY_FILE_NO alias (MatchKey ALIAS|LEGACY_REGISTRY_FILE_NO||). If the file cannot be written a VOID row accounts for the number and the reply is 500; if no number can be claimed, 503.
- move_file (T12): moves a registry file to toOrgKey. The file (REG: or its spo: locator) is found in DGO_RegistryFiles; its CurrentHolder changes by the Guarded Update (Rev + 1) and DGO_FileMovements gets a MOVED event: SubjectKey and RootKey REG:, RootRef the FILE reference, FromOrgKey (the holder before), ToOrgKey, MovementAction (as sent, else MOVED), RevAfter. An unknown file is 404; a file changed three times during the move is 409.
- link (§6, T12): writes a DGO_RecordLinks row: FromKey, LinkType, ToKey, LinkMatchKey || (unique), EntityKey, IdState ACTIVE, Rev 1, CreatedByRequest. A link that already exists is returned (existing: true), not written twice.
- void (§9): sets the target's IdState to VOID by the Guarded Update (Rev + 1) and writes VoidReason where the list has it (Portal Registry). The row is kept and its reference is never

reissued. The audit event VOIDED carries the reason. A target already void is answered unchanged.

- reparent (§9): moves an attachment, task, minute or approval under a new parent. ParentKey becomes parentKey; RootKey becomes parentKey when the new parent is a REC or CAS, else stays as it was. Guarded Update (Rev + 1); audit event REPARENTED with the old and new parent.
- merge (§9 merge procedure, T9; Registry Manager only): absorbs record A (target) into survivor B. 1. LNK A MERGED_INTO B. 2. A's attachments (Portal Attachments, ParentKey REC:A) and tasks (Global Tracking Queue, ParentKey or RootKey REC:A) are re-parented to B, each by the Guarded Update with RootRef B's reference and an audit event REPARENTED. 3. A: IdState MERGED and, where the list has it, SupersededBy REC:B, by the Guarded Update (Rev + 1). 4. Audit event MERGED on A and on B. A's reference resolves to B from then on. Either record unknown is 404; a record already merged or void cannot be merged (409).
- The reply (Compose_Outcome) takes the result of whichever new operation ran, as it does for the live ones.
- ITEM-25 (owner decision 2026-10-08, on the needs assessment): dispatch, dispatchoutbound and archivereference are served by transitionstatus instead of answering 501. A letter goes out through the DGO Correspondence Mailer; dispatch records the document as Treated (the Status column offers Treated and Not Treated). archivereference records it as Treated and sets its IdState to ARCHIVED (the catalogue's lifecycle state), before the unchanged check so an already treated document is still archived. The caller's role must allow the dispatch route (dispatch) or the archive route (archive). The audit row names the operation asked for.
- Gap found 2026-10-10: a task's assignee whose role lacks the My Work route (Read-only Viewer, Executive: 852 of 854 people) could acknowledge a task but never start, update or complete it, because update-task required the orchestrator route at the gate. The gate now lets update-task through for any active role, and the update-task case decides once the task is read: a role with the orchestrator route (or "*") may update any task, as before; anyone else only a task assigned to them, and is refused 403 otherwise.
- ITEM-25: the activity lifecycle steps (activity-archive, activity-siwes, activity-nysc :create-queue-record, :set-reference-id, :update-activity) are served by IP_Activity_Lifecycle_Endpoint (ACTIVITY_LIFECYCLE); here they are refused as unknown (400), not as recognised-but-unserved (501).
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Compose_Dynamic_Replay_Meta`, `Compose_Result_Dynamic_403_Task`, `Condition_Dynamic_Archive_IdState`, `Update_Dynamic_Archive_IdState`,

`Get_Dynamic_Register_Replay`, `Condition_Dynamic_Register_Replay`, `Compose_Result_Dynamic_200_Register_Replay`, `Compose_EntityKey_Record`, `Compose_Claim_Registry_Year`, `Get_Claim_Registry_Floor`, `Compose_Claim_Registry_Floor`, `Compose_Claim_Registry_Count`, `Scope_Claim_Registry_1`, `Get_Counter_Registry_1`, `Condition_Counter_Registry_1_Exists`, `Claim_Counter_Registry_1`, `Create_Counter_Registry_1`, `Scope_Claim_Registry_2`, `Delay_Claim_Registry_2`, `Get_Counter_Registry_2`, `Condition_Counter_Registry_2_Exists`, `Claim_Counter_Registry_2`, `Create_Counter_Registry_2`, `Scope_Claim_Registry_3`, `Delay_Claim_Registry_3`, `Get_Counter_Registry_3`, `Condition_Counter_Registry_3_Exists`, `Claim_Counter_Registry_3`, `Create_Counter_Registry_3`, `Scope_Claim_Registry_4`, `Delay_Claim_Registry_4`, `Get_Counter_Registry_4`, `Condition_Counter_Registry_4_Exists`, `Claim_Counter_Registry_4`, `Create_Counter_Registry_4`, `Scope_Claim_Registry_5`, `Delay_Claim_Registry_5`, `Get_Counter_Registry_5`, `Condition_Counter_Registry_5_Exists`, `Claim_Counter_Registry_5`, `Create_Counter_Registry_5`, `Scope_Claim_Registry_6`, `Delay_Claim_Registry_6`, `Get_Counter_Registry_6`, `Condition_Counter_Registry_6_Exists`, `Claim_Counter_Registry_6`, `Create_Counter_Registry_6`, `Scope_Claim_Registry_7`, `Delay_Claim_Registry_7`, `Get_Counter_Registry_7`, `Condition_Counter_Registry_7_Exists`, `Claim_Counter_Registry_7`, `Create_Counter_Registry_7`, `Scope_Claim_Registry_8`, `Delay_Claim_Registry_8`, `Get_Counter_Registry_8`, `Condition_Counter_Registry_8_Exists`, `Claim_Counter_Registry_8`, `Create_Counter_Registry_8`, `Scope_Claim_Registry_9`, `Delay_Claim_Registry_9`, `Get_Counter_Registry_9`, `Condition_Counter_Registry_9_Exists`, `Claim_Counter_Registry_9`, `Create_Counter_Registry_9`, `Scope_Claim_Registry_10`, `Delay_Claim_Registry_10`, `Get_Counter_Registry_10`, `Condition_Counter_Registry_10_Exists`, `Claim_Counter_Registry_10`, `Create_Counter_Registry_10`, `Compose_Claim_Registry_Number`, `Compose_Claim_Registry`, `Select_Claim_Registry_References`, `Condition_Dynamic_Register_Claimed`, `Create_Dynamic_Register_Record`, `Create_Void_Dynamic_Register`, `Compose_Result_Dynamic_500_Register`, `Compose_MatchKey_Dynamic_Register_Locator`, `Get_MatchKey_Dynamic_Register_Locator`, `Condition_Dynamic_Register_Locator`, `Create_Dynamic_Register_Locator`, `Compose_Dynamic_Register_Data`, `Create_Dynamic_Register_Audit`, `Compose_Result_Dynamic_200_Register`, `Compose_Result_Dynamic_503_Register`, `Condition_Dynamic_Upsert_Change`, `Get_Dynamic_Upsert_Row`, `Condition_Dynamic_Upsert_Found`, `Scope_Update_Dynamic_Upsert_1`, `Get_Row_Dynamic_Upsert_1`, `Merge_Row_Dynamic_Upsert_1`, `Compose_Rev_Dynamic_Upsert_1`, `Scope_Update_Dynamic_Upsert_2`, `Get_Row_Dynamic_Upsert_2`, `Merge_Row_Dynamic_Upsert_2`, `Compose_Rev_Dynamic_Upsert_2`, `Scope_Update_Dynamic_Upsert_3`, `Get_Row_Dynamic_Upsert_3`, `Merge_Row_Dynamic_Upsert_3`, `Compose_Rev_Dynamic_Upsert_3`, `Compose_Updated_Dynamic_Upsert`, `Condition_Dynamic_Upsert_Updated`, `Compose_Dynamic_Upsert_Change_Data`, `Create_Dynamic_Upsert_Change_Audit`, `Compose_Result_Dynamic_200_Upsert_Change`, `Compose_Result_Dynamic_409_Upsert`, `Compose_Result_Dynamic_404_Upsert`, `Get_Dynamic_Upsert_Replay`, `Get_Dynamic_Upsert_Parent`, `Compose_Dynamic_Upsert_State`, `Switch_Dynamic_Upsert_State`, `Compose_Result_Dynamic_200_Upsert_Replay`, `Compose_Result_Dynamic_404_Upsert_Parent`, `Compose_EntityKey_Upsert`, `Create_Dynamic_Upsert_Row`, `Compose_MatchKey_Dynamic_Upsert_Alias`, `Get_MatchKey_Dynamic_Upsert_Alias`, `Condition_Dynamic_Upsert_Alias`, `Create_Dynamic_Upsert_Alias`, `Compose_Dynamic_Upsert_Data`, `Create_Dynamic_Upsert_Audit`,

`Compose_Result_Dynamic_200_Upsert`, `Compose_Result_Dynamic_422_Upsert`, `Get_Dynamic_File_Replay`, `Condition_Dynamic_File_Replay`, `Compose_Result_Dynamic_200_File_Replay`, `Compose_EntityKey_File`, `Compose_Claim_File_Year`, `Get_Claim_File_Floor`, `Compose_Claim_File_Floor`, `Compose_Claim_File_Count`, `Scope_Claim_File_1`, `Get_Counter_File_1`, `Condition_Counter_File_1_Exists`, `Claim_Counter_File_1`, `Create_Counter_File_1`, `Scope_Claim_File_2`, `Delay_Claim_File_2`, `Get_Counter_File_2`, `Condition_Counter_File_2_Exists`, `Claim_Counter_File_2`, `Create_Counter_File_2`, `Scope_Claim_File_3`, `Delay_Claim_File_3`, `Get_Counter_File_3`, `Condition_Counter_File_3_Exists`, `Claim_Counter_File_3`, `Create_Counter_File_3`, `Scope_Claim_File_4`, `Delay_Claim_File_4`, `Get_Counter_File_4`, `Condition_Counter_File_4_Exists`, `Claim_Counter_File_4`, `Create_Counter_File_4`, `Scope_Claim_File_5`, `Delay_Claim_File_5`, `Get_Counter_File_5`, `Condition_Counter_File_5_Exists`, `Claim_Counter_File_5`, `Create_Counter_File_5`, `Scope_Claim_File_6`, `Delay_Claim_File_6`, `Get_Counter_File_6`, `Condition_Counter_File_6_Exists`, `Claim_Counter_File_6`, `Create_Counter_File_6`, `Scope_Claim_File_7`, `Delay_Claim_File_7`, `Get_Counter_File_7`, `Condition_Counter_File_7_Exists`, `Claim_Counter_File_7`, `Create_Counter_File_7`, `Scope_Claim_File_8`, `Delay_Claim_File_8`, `Get_Counter_File_8`, `Condition_Counter_File_8_Exists`, `Claim_Counter_File_8`, `Create_Counter_File_8`, `Scope_Claim_File_9`, `Delay_Claim_File_9`, `Get_Counter_File_9`, `Condition_Counter_File_9_Exists`, `Claim_Counter_File_9`, `Create_Counter_File_9`, `Scope_Claim_File_10`, `Delay_Claim_File_10`, `Get_Counter_File_10`, `Condition_Counter_File_10_Exists`, `Claim_Counter_File_10`, `Create_Counter_File_10`, `Compose_Claim_File_Number`, `Compose_Claim_File`, `Select_Claim_File_References`, `Condition_Dynamic_File_Claimed`, `Create_Dynamic_File`, `Create_Void_Dynamic_File`, `Compose_Result_Dynamic_500_File`, `Create_Dynamic_File_Opened`, `Compose_MatchKey_Dynamic_File_Alias`, `Get_MatchKey_Dynamic_File_Alias`, `Condition_Dynamic_File_Alias`, `Create_Dynamic_File_Alias`, `Compose_Dynamic_File_Data`, `Create_Dynamic_File_Audit`, `Compose_Result_Dynamic_200_File`, `Compose_Result_Dynamic_503_File`, `Compose_T_Move_Raw`, `Compose_T_Move`, `Select_T_Move_Bound`, `Compose_T_Move_Dataset`, `Get_T_Move_Row`, `Get_T_Move_Row_Ops`, `Compose_Row_Move`, `Compose_Row_Move_Keys`, `Condition_Dynamic_Move_Found`, `Scope_Update_Dynamic_Move_1`, `Get_Row_Dynamic_Move_1`, `Merge_Row_Dynamic_Move_1`, `Compose_Rev_Dynamic_Move_1`, `Scope_Update_Dynamic_Move_2`, `Get_Row_Dynamic_Move_2`, `Merge_Row_Dynamic_Move_2`, `Compose_Rev_Dynamic_Move_2`, `Scope_Update_Dynamic_Move_3`, `Get_Row_Dynamic_Move_3`, `Merge_Row_Dynamic_Move_3`, `Compose_Rev_Dynamic_Move_3`, `Compose_Updated_Dynamic_Move`, `Condition_Dynamic_Move_Updated`, `Create_Dynamic_Move_Event`, `Compose_Dynamic_Move_Data`, `Create_Dynamic_Move_Audit`, `Compose_Result_Dynamic_200_Move`, `Compose_Result_Dynamic_409_Move`, `Compose_Result_Dynamic_404_Move`, `Compose_Dynamic_Link_Key`, `Get_Dynamic_Link_Existing`, `Condition_Dynamic_Link_Exists`, `Compose_Result_Dynamic_200_Link_Existing`, `Compose_EntityKey_Link`, `Create_Dynamic_Link`, `Compose_Dynamic_Link_Data`, `Create_Dynamic_Link_Audit`, `Compose_Result_Dynamic_200_Link`, `Compose_Result_Dynamic_422_Link`, `Compose_T_Void_Raw`, `Compose_T_Void`, `Select_T_Void_Bound`, `Compose_T_Void_Dataset`, `Get_T_Void_Row`, `Get_T_Void_Row_Ops`, `Compose_Row_Void`, `Compose_Row_Void_Keys`, `Compose_Dynamic_Void_State`, `Switch_Dynamic_Void_State`, `Compose_Result_Dynamic_404_Void`, `Compose_Result_Dynamic_200_Void_Unchanged`, `Scope_Update_Dynamic_Void_1`,

`Get_Row_Dynamic_Void_1`, `Merge_Row_Dynamic_Void_1`, `Compose_Rev_Dynamic_Void_1`, `Scope_Update_Dynamic_Void_2`, `Get_Row_Dynamic_Void_2`, `Merge_Row_Dynamic_Void_2`, `Compose_Rev_Dynamic_Void_2`, `Scope_Update_Dynamic_Void_3`, `Get_Row_Dynamic_Void_3`, `Merge_Row_Dynamic_Void_3`, `Compose_Rev_Dynamic_Void_3`, `Compose_Updated_Dynamic_Void`, `Condition_Dynamic_Void_Updated`, `Compose_Dynamic_Void_Data`, `Create_Dynamic_Void_Audit`, `Compose_Result_Dynamic_200_Void`, `Compose_Result_Dynamic_409_Void`, `Compose_T_Reparent_Raw`, `Compose_T_Reparent`, `Select_T_Reparent_Bound`, `Compose_T_Reparent_Dataset`, `Get_T_Reparent_Row`, `Get_T_Reparent_Row_Ops`, `Compose_Row_Reparent`, `Compose_Row_Reparent_Keys`, `Condition_Dynamic_Reparent_Found`, `Scope_Update_Dynamic_Reparent_1`, `Get_Row_Dynamic_Reparent_1`, `Merge_Row_Dynamic_Reparent_1`, `Compose_Rev_Dynamic_Reparent_1`, `Scope_Update_Dynamic_Reparent_2`, `Get_Row_Dynamic_Reparent_2`, `Merge_Row_Dynamic_Reparent_2`, `Compose_Rev_Dynamic_Reparent_2`, `Scope_Update_Dynamic_Reparent_3`, `Get_Row_Dynamic_Reparent_3`, `Merge_Row_Dynamic_Reparent_3`, `Compose_Rev_Dynamic_Reparent_3`, `Compose_Updated_Dynamic_Reparent`, `Condition_Dynamic_Reparent_Updated`, `Compose_Dynamic_Reparent_Data`, `Create_Dynamic_Reparent_Audit`, `Compose_Result_Dynamic_200_Reparent`, `Compose_Result_Dynamic_409_Reparent`, `Compose_Result_Dynamic_404_Reparent`, `Compose_T_MergeA_Raw`, `Compose_T_MergeA`, `Select_T_MergeA_Bound`, `Compose_T_MergeA_Dataset`, `Get_T_MergeA_Row`, `Get_T_MergeA_Row_Ops`, `Compose_Row_MergeA`, `Compose_Row_MergeA_Keys`, `Compose_T_MergeB_Raw`, `Compose_T_MergeB`, `Select_T_MergeB_Bound`, `Compose_T_MergeB_Dataset`, `Get_T_MergeB_Row`, `Get_T_MergeB_Row_Ops`, `Compose_Row_MergeB`, `Compose_Row_MergeB_Keys`, `Compose_Dynamic_Merge_State`, `Switch_Dynamic_Merge_State`, `Compose_Result_Dynamic_404_Merge`, `Compose_Result_Dynamic_409_Merge_State`, `Create_Dynamic_Merge_Link`, `Scope_Dynamic_Merge_Children`, `Get_Dynamic_Merge_Attachments`, `Apply_to_each_Merge_Attachments`, `Scope_Update_Merge_Attachments_1`, `Get_Row_Merge_Attachments_1`, `Merge_Row_Merge_Attachments_1`, `Compose_Rev_Merge_Attachments_1`, `Scope_Update_Merge_Attachments_2`, `Get_Row_Merge_Attachments_2`, `Merge_Row_Merge_Attachments_2`, `Compose_Rev_Merge_Attachments_2`, `Scope_Update_Merge_Attachments_3`, `Get_Row_Merge_Attachments_3`, `Merge_Row_Merge_Attachments_3`, `Compose_Rev_Merge_Attachments_3`, `Compose_Updated_Merge_Attachments`, `Create_Dynamic_Merge_Attachments_Audit`, `Get_Dynamic_Merge_Tasks`, `Apply_to_each_Merge_Tasks`, `Scope_Update_Merge_Tasks_1`, `Get_Row_Merge_Tasks_1`, `Merge_Row_Merge_Tasks_1`, `Compose_Rev_Merge_Tasks_1`, `Scope_Update_Merge_Tasks_2`, `Get_Row_Merge_Tasks_2`, `Merge_Row_Merge_Tasks_2`, `Compose_Rev_Merge_Tasks_2`, `Scope_Update_Merge_Tasks_3`, `Get_Row_Merge_Tasks_3`, `Merge_Row_Merge_Tasks_3`, `Compose_Rev_Merge_Tasks_3`, `Compose_Updated_Merge_Tasks`, `Create_Dynamic_Merge_Tasks_Audit`, `Scope_Update_Dynamic_Merge_1`, `Get_Row_Dynamic_Merge_1`, `Merge_Row_Dynamic_Merge_1`, `Compose_Rev_Dynamic_Merge_1`, `Scope_Update_Dynamic_Merge_2`, `Get_Row_Dynamic_Merge_2`, `Merge_Row_Dynamic_Merge_2`, `Compose_Rev_Dynamic_Merge_2`, `Scope_Update_Dynamic_Merge_3`, `Get_Row_Dynamic_Merge_3`, `Merge_Row_Dynamic_Merge_3`, `Compose_Rev_Dynamic_Merge_3`, `Compose_Updated_Dynamic_Merge`, `Condition_Dynamic_Merge_Updated`, `Compose_Dynamic_Merge_Data`, `Create_Dynamic_Merge_Audit`, `Create_Dynamic_Merge_Survivor_Audit`, `Compose_Result_Dynamic_200_Merge`, `Compose_Result_Dynamic_409_Merge`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`,

`Compose_Dynamic_Actor_Key`, `Compose_Dynamic_Hash`, `Get_Dynamic_Gate_Proof_Mode`, `Compose_Dynamic_Gate_Proof_Token`, `Get_Dynamic_Gate_Proof`, `Compose_Dynamic_Gate_Identity`, `Compose_Dynamic_Gate_Caller`, `Compose_Invalid_RMD`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: none.

Test after pasting. From Folio (or any client signed in as an active directory user), send: register {subject, channel email, locatorKind EXO_INTERNET_MESSAGE_ID, locatorContainer , locatorValue } → a NITDA-YYYY-NNNNN and a REC key, a Portal Registry row and a DGO_IdentityMap LOCATOR row; upsert_record {entityType MIN, parentKey TSK:<a task's EntityKey>, text} → a Task_Comments row under that task; open_file {title} → FILE-YYYY-NNNNN; link {fromKey REC:..., toKey REG:..., linkType FILED_IN}; move_file twice {target REG:..., toOrgKey} → two MOVED rows in DGO_FileMovements (T12). Send any of them again with the same clientRequestHash: the first result comes back with replayed true and nothing new is written (T11). merge two test records as a Registry Manager (T9).

USER_ADMIN

- S8 / ID-19: the Request ID is the flow's own guid(). It was the client's X-Correlation-ID or requestId whenever one was sent; that value is now kept only as clientCorrelationId (sanitised, 64 characters).
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. With the switch true and no valid proof, the gate answers 401. The role is read from the directory row's Role choice, which the flow read as text and so never matched a RoleId.
- The request may now carry, at its top level or under payload, the fields the identity operations read: entityType, entityKey, target, survivor, fromKey, toKey, linkType, title, text, localKey, parentKey, reason, toOrgKey, movementAction, subject, senderEmail, channel, locatorKind, locatorContainer, locatorValue, legacyNumber, decision and clientRequestHash.
- The gate serves the new operations (register, upsert-record, open-file, move-file, link, void, reparent, merge; underscores are read as hyphens, so upsert_record is upsert-record). upsert-record leaves the recognised-but-unserved set. Routes the caller's role must allow: register and link need correspondence; upsert-record needs briefs, meetings, projects, approvals or orchestrator (minutes) by entity type; open-file, move-file, void and reparent need registry; merge needs registry-merge (Registry Manager only, §9: add "registry-merge" to that role's AllowedRoutesJson; "*" also passes).
- Each new operation is checked before anything is written; a request that fails is refused with 400 and the reason. register: a subject. upsert-record: entity type MIN, BRF, MTG, PRJ, APR or RMD (a reminder: assignedTo and dueAt, written once); a title (or, for a minute, its text and its task as parentKey TSK:) unless entityKey names the row to change. open-file: a title. move-file: the file as target and toOrgKey. link: fromKey and toKey as Qualified Keys and one of the 11 link types. void: a target and a reason. reparent: a target ATT, TSK, MIN or APR Qualified Key and the new parentKey. merge: the absorbed record as target and the survivor, two different REC

Qualified Keys. A target is a Qualified Key TYPE: or spo:/ (B10); a bare item ID, RefIDD or title is refused.

- T11 / B3: a replayed request (the same idempotency key; Folio sends its clientRequestHash as that key) returns the first result unchanged. Every new operation stores its result on its audit event, and the replay answer is that result; older events, which hold none, keep the replay answer they had.
- ITEM-81 (from the hardened scope): an idempotency key already used for a different operation answers 409 IDEMPOTENCY_CONFLICT and nothing is replayed or written; only the same request gets the first result back.
- register (§8.2, S7): registers a record received outside the portal (Folio, e-mail, library). A request whose clientRequestHash Portal Registry already holds returns that record. Otherwise it claims the next NITDA-YYYY-NNNNN (the kit claim, floored at the register's highest number) and writes the Portal Registry row: Title and ReferenceId, Subject, Channel (one of the catalogue's channel codes, else internal), CorrespondenceType Incoming, SenderEmail, Status received, EntityKey, IdState ACTIVE, Rev 1, Direction In, CorrelationId, CreatedByRequest, ClientRequestHash and, when sent, SourceLocator. A locator (locatorKind EXO_INTERNET_MESSAGE_ID, SPO_ITEM, ODB_ITEM or PAPER_FILE, with locatorContainer and locatorValue; for an e-mail, the registry mailbox and the internetMessageId, T8) is written to DGO_IdentityMap as a LOCATOR entry: System from the catalogue, MatchKey LOCATOR||| (§10.2), not written again when that key exists. If the row cannot be written a VOID row accounts for the number (Q2) and the reply is 500; if no number can be claimed, 503. The audit event REGISTERED carries REC:.
- upsert_record (§7, §13): creates or changes a minute, brief, meeting, project or approval. With entityKey it changes that row's Title (a minute's Description, an approval's Decision) by the kit's Guarded Update (Rev + 1); without, it creates the row with a new Entity Key, IdState ACTIVE, Rev 1, CreatedByRequest and ClientRequestHash (CorrelationId where the list has it), after returning the existing row if its clientRequestHash is already there. A minute (Task_Comments) is created under its task: ParentKey TSK:, RootKey and RootRef from the task row, RefIDD the task's document, Description the text, AuthorKey USR:; an unknown task is 404. An approval takes Decision (Pending unless sent) and, when parentKey is given, ParentKey and RootKey. When Folio sends its own local key (localKey), DGO_IdentityMap gets a FOLIO_LOCAL_KEY alias, System runtime, MatchKey ALIAS|FOLIO_LOCAL_KEY|| (§12.3 4a).
- open_file (§8.2, T12): opens a registry file. A request whose clientRequestHash DGO_RegistryFiles already holds returns that file. Otherwise it claims the next FILE-YYYY-NNNNN (the kit claim, counter FILE-) and writes the file: Title, RegistryFileRef, CurrentHolder (toOrgKey, else the caller), EntityKey, IdState ACTIVE, Rev 1, CorrelationId, CreatedByRequest, ClientRequestHash; then the OPENED movement in DGO_FileMovements; and, when Folio sends the file's old DGO/YYYY/NNNNNN number (legacyNumber), a LEGACY_REGISTRY_FILE_NO alias (MatchKey ALIAS|LEGACY_REGISTRY_FILE_NO||). If the file cannot be written a VOID row accounts for the number and the reply is 500; if no number can be claimed, 503.
- move_file (T12): moves a registry file to toOrgKey. The file (REG: or its spo: locator) is found in DGO_RegistryFiles; its CurrentHolder changes by the Guarded Update (Rev + 1) and

DGO_FileMovements gets a MOVED event: SubjectKey and RootKey REG:, RootRef the FILE reference, FromOrgKey (the holder before), ToOrgKey, MovementAction (as sent, else MOVED), RevAfter. An unknown file is 404; a file changed three times during the move is 409.

- link (§6, T12): writes a DGO_RecordLinks row: FromKey, LinkType, ToKey, LinkMatchKey || (unique), EntityKey, IdState ACTIVE, Rev 1, CreatedByRequest. A link that already exists is returned (existing: true), not written twice.
- void (§9): sets the target's IdState to VOID by the Guarded Update (Rev + 1) and writes VoidReason where the list has it (Portal Registry). The row is kept and its reference is never reissued. The audit event VOIDED carries the reason. A target already void is answered unchanged.
- reparent (§9): moves an attachment, task, minute or approval under a new parent. ParentKey becomes parentKey; RootKey becomes parentKey when the new parent is a REC or CAS, else stays as it was. Guarded Update (Rev + 1); audit event REPARENTED with the old and new parent.
- merge (§9 merge procedure, T9; Registry Manager only): absorbs record A (target) into survivor B. 1. LNK A MERGED_INTO B. 2. A's attachments (Portal Attachments, ParentKey REC:A) and tasks (Global Tracking Queue, ParentKey or RootKey REC:A) are re-parented to B, each by the Guarded Update with RootRef B's reference and an audit event REPARENTED. 3. A: IdState MERGED and, where the list has it, SupersededBy REC:B, by the Guarded Update (Rev + 1). 4. Audit event MERGED on A and on B. A's reference resolves to B from then on. Either record unknown is 404; a record already merged or void cannot be merged (409).
- ITEM-25 user-admin (needs assessment 2026-10-08: the User Administration screen's changes queued on the device and never reached DGO_UserDirectory, so a person could be added or given a role only in SharePoint itself): user-admin:create-user, update-user, assign-role and disable-user are served. The caller's role must allow the user-admin route. The person is found by e-mail. create-user writes a new directory row (UserId, EntityKey, IdState ACTIVE, Rev 1, CreatedAt/By) and is refused 409 when the address is already there; the others change the row by Guarded Update (Rev + 1) and are refused 404 when it is not. Role, Persona and Status must be the directory's own choices (400). Only a systemAdmin grants systemAdmin or changes a systemAdmin's row (403). Each change is audited as user: with the officer as actor.
- The reply (Compose_Outcome) takes the result of whichever new operation ran, as it does for the live ones.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Compose_Dynamic_Replay_Meta`, `Get_Dynamic_User_Row`, `Compose_Dynamic_User_State`, `Switch_Dynamic_User_State`, `Compose_Result_Dynamic_403_User`,

`Compose_Result_Dynamic_409_User_Exists`, `Compose_Result_Dynamic_404_User`, `Compose_Dynamic_New_User_Id`, `Create_Dynamic_User`, `Condition_Dynamic_User_Created`, `Create_Dynamic_User_Audit`, `Compose_Result_Dynamic_200_User_Created`, `Compose_Result_Dynamic_422_User`, `Scope_Update_Dynamic_User_1`, `Get_Row_Dynamic_User_1`, `Merge_Row_Dynamic_User_1`, `Compose_Rev_Dynamic_User_1`, `Scope_Update_Dynamic_User_2`, `Get_Row_Dynamic_User_2`, `Merge_Row_Dynamic_User_2`, `Compose_Rev_Dynamic_User_2`, `Scope_Update_Dynamic_User_3`, `Get_Row_Dynamic_User_3`, `Merge_Row_Dynamic_User_3`, `Compose_Rev_Dynamic_User_3`, `Compose_Updated_Dynamic_User`, `Condition_Dynamic_User_Updated`, `Create_Dynamic_User_Change_Audit`, `Compose_Result_Dynamic_200_User_Changed`, `Compose_Result_Dynamic_409_User`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`, `Compose_Dynamic_Actor_Key`, `Compose_Dynamic_Hash`, `Get_Dynamic_Gate_Proof_Mode`, `Compose_Dynamic_Gate_Proof_Token`, `Get_Dynamic_Gate_Proof`, `Compose_Dynamic_Gate_Identity`, `Compose_Dynamic_Gate_Caller`, `Compose_Invalid_RMD`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: `Get_Dynamic_Document`, `Compose_Dynamic_Document_State`, `Switch_Dynamic_Document_State`, `Update_Dynamic_Flag`, `Compose_Dynamic_Flag_Meta`, `Create_Dynamic_Flag_Audit`, `Compose_Result_Dynamic_200_Flag`, `Compose_Result_Dynamic_422_Flag`, `Compose_Result_Dynamic_404_Document`, `Compose_Result_Dynamic_409_Document`, `Get_Dynamic_Task`, `Compose_Dynamic_Task_State`, `Switch_Dynamic_Task_State`, `Update_Dynamic_Task`, `Compose_Dynamic_Task_Meta`, `Create_Dynamic_Task_Audit`, `Compose_Result_Dynamic_200_Task`, `Compose_Result_Dynamic_422_Task`, `Compose_Result_Dynamic_404_Task`, `Compose_Result_Dynamic_409_Task`, `Get_Dynamic_Status_Document`, `Compose_Dynamic_Status_State`, `Switch_Dynamic_Status_State`, `Condition_Dynamic_Status_Changed`, `Compose_Result_Dynamic_200_Status_Unchanged`, `Update_Dynamic_Status`, `Compose_Dynamic_Status_Meta`, `Create_Dynamic_Status_Audit`, `Compose_Result_Dynamic_200_Status`, `Compose_Result_Dynamic_422_Status`, `Compose_Result_Dynamic_404_Status`, `Compose_Result_Dynamic_409_Status`, `Compose_Dynamic_Event_Meta`, `Create_Dynamic_Event_Audit`, `Compose_Result_Dynamic_200_Event`, `Compose_Result_Dynamic_503_Event`.

Test after pasting. In Folio, User Administration: add a person, change their role, disable them. Each change appears in DGO_UserDirectory at once (Rev rising) with an audit row user:. A user administrator granting systemAdmin is refused (403).

ACTIVITY_LIFECYCLE

- S8 / ID-19: the Request ID is the flow's own guid(). It was the client's X-Correlation-ID or requestId whenever one was sent; that value is now kept only as clientCorrelationId (sanitised, 64 characters).
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. With the switch true and no valid proof, the gate answers 401. The role is read from the directory row's Role choice, which the flow read as text and so never matched a RoleId.

- The request may now carry, at its top level or under payload, the fields the identity operations read: entityType, entityKey, target, survivor, fromKey, toKey, linkType, title, text, localKey, parentKey, reason, toOrgKey, movementAction, subject, senderEmail, channel, locatorKind, locatorContainer, locatorValue, legacyNumber, decision and clientRequestHash.
- The gate serves the new operations (register, upsert-record, open-file, move-file, link, void, reparent, merge; underscores are read as hyphens, so upsert_record is upsert-record). upsert-record leaves the recognised-but-unserved set. Routes the caller's role must allow: register and link need correspondence; upsert-record needs briefs, meetings, projects, approvals or orchestrator (minutes) by entity type; open-file, move-file, void and reparent need registry; merge needs registry-merge (Registry Manager only, §9: add "registry-merge" to that role's AllowedRoutesJson; "*" also passes).
- Each new operation is checked before anything is written; a request that fails is refused with 400 and the reason. register: a subject. upsert-record: entity type MIN, BRF, MTG, PRJ, APR or RMD (a reminder: assignedTo and dueAt, written once); a title (or, for a minute, its text and its task as parentKey TSK:) unless entityKey names the row to change. open-file: a title. move-file: the file as target and toOrgKey. link: fromKey and toKey as Qualified Keys and one of the 11 link types. void: a target and a reason. reparent: a target ATT, TSK, MIN or APR Qualified Key and the new parentKey. merge: the absorbed record as target and the survivor, two different REC Qualified Keys. A target is a Qualified Key TYPE: or spo:/ (B10); a bare item ID, RefIDD or title is refused.
- T11 / B3: a replayed request (the same idempotency key; Folio sends its clientRequestHash as that key) returns the first result unchanged. Every new operation stores its result on its audit event, and the replay answer is that result; older events, which hold none, keep the replay answer they had.
- ITEM-81 (from the hardened scope): an idempotency key already used for a different operation answers 409 IDEMPOTENCY_CONFLICT and nothing is replayed or written; only the same request gets the first result back.
- register (§8.2, S7): registers a record received outside the portal (Folio, e-mail, library). A request whose clientRequestHash Portal Registry already holds returns that record. Otherwise it claims the next NITDA-YYYY-NNNNN (the kit claim, floored at the register's highest number) and writes the Portal Registry row: Title and ReferenceId, Subject, Channel (one of the catalogue's channel codes, else internal), CorrespondenceType Incoming, SenderEmail, Status received, EntityKey, IdState ACTIVE, Rev 1, Direction In, CorrelationId, CreatedByRequest, ClientRequestHash and, when sent, SourceLocator. A locator (locatorKind EXO_INTERNET_MESSAGE_ID, SPO_ITEM, ODB_ITEM or PAPER_FILE, with locatorContainer and locatorValue; for an e-mail, the registry mailbox and the internetMessageId, T8) is written to DGO_IdentityMap as a LOCATOR entry: System from the catalogue, MatchKey LOCATOR||| (§10.2), not written again when that key exists. If the row cannot be written a VOID row accounts for the number (Q2) and the reply is 500; if no number can be claimed, 503. The audit event REGISTERED carries REC:.
- upsert_record (§7, §13): creates or changes a minute, brief, meeting, project or approval. With entityKey it changes that row's Title (a minute's Description, an approval's Decision) by the kit's Guarded Update (Rev + 1); without, it creates the row with a new Entity Key, IdState ACTIVE,

Rev 1, CreatedByRequest and ClientRequestHash (CorrelationId where the list has it), after returning the existing row if its clientRequestHash is already there. A minute (Task_Comments) is created under its task: ParentKey TSK:, RootKey and RootRef from the task row, RefIDD the task's document, Description the text, AuthorKey USR:; an unknown task is 404. An approval takes Decision (Pending unless sent) and, when parentKey is given, ParentKey and RootKey. When Folio sends its own local key (localKey), DGO_IdentityMap gets a FOLIO_LOCAL_KEY alias, System runtime, MatchKey ALIAS|FOLIO_LOCAL_KEY|| (§12.3 4a).

- open_file (§8.2, T12): opens a registry file. A request whose clientRequestHash DGO_RegistryFiles already holds returns that file. Otherwise it claims the next FILE-YYYY-NNNNN (the kit claim, counter FILE-) and writes the file: Title, RegistryFileRef, CurrentHolder (toOrgKey, else the caller), EntityKey, IdState ACTIVE, Rev 1, CorrelationId, CreatedByRequest, ClientRequestHash; then the OPENED movement in DGO_FileMovements; and, when Folio sends the file's old DGO/YYYY/NNNNNN number (legacyNumber), a LEGACY_REGISTRY_FILE_NO alias (MatchKey ALIAS|LEGACY_REGISTRY_FILE_NO||). If the file cannot be written a VOID row accounts for the number and the reply is 500; if no number can be claimed, 503.
- move_file (T12): moves a registry file to toOrgKey. The file (REG: or its spo: locator) is found in DGO_RegistryFiles; its CurrentHolder changes by the Guarded Update (Rev + 1) and DGO_FileMovements gets a MOVED event: SubjectKey and RootKey REG:, RootRef the FILE reference, FromOrgKey (the holder before), ToOrgKey, MovementAction (as sent, else MOVED), RevAfter. An unknown file is 404; a file changed three times during the move is 409.
- link (§6, T12): writes a DGO_RecordLinks row: FromKey, LinkType, ToKey, LinkMatchKey || (unique), EntityKey, IdState ACTIVE, Rev 1, CreatedByRequest. A link that already exists is returned (existing: true), not written twice.
- void (§9): sets the target's IdState to VOID by the Guarded Update (Rev + 1) and writes VoidReason where the list has it (Portal Registry). The row is kept and its reference is never reissued. The audit event VOIDED carries the reason. A target already void is answered unchanged.
- reparent (§9): moves an attachment, task, minute or approval under a new parent. ParentKey becomes parentKey; RootKey becomes parentKey when the new parent is a REC or CAS, else stays as it was. Guarded Update (Rev + 1); audit event REPARENTED with the old and new parent.
- merge (§9 merge procedure, T9; Registry Manager only): absorbs record A (target) into survivor B. 1. LNK A MERGED_INTO B. 2. A's attachments (Portal Attachments, ParentKey REC:A) and tasks (Global Tracking Queue, ParentKey or RootKey REC:A) are re-parented to B, each by the Guarded Update with RootRef B's reference and an audit event REPARENTED. 3. A: IdState MERGED and, where the list has it, SupersededBy REC:B, by the Guarded Update (Rev + 1). 4. Audit event MERGED on A and on B. A's reference resolves to B from then on. Either record unknown is 404; a record already merged or void cannot be merged (409).
- The reply (Compose_Outcome) takes the result of whichever new operation ran, as it does for the live ones.
- ITEM-25 (owner decision 2026-10-10): activity-archive, activity-siwes and activity-nysc are served, as the Canvas Activities screen did them, in the three steps Folio sends. The activity is

read from DGO DIGITAL OPS by its item ID (404 if absent). create-queue-record writes a Global Tracking Queue row built from that activity, not from the request: Title, ActivityTrackingID (its RefIDD), RefIDD and RefIDDN (its ID), AttachmentLink, StartDate, DueDate and Acknowledgement Due Date today, AssignedBy DGS OFFICE, AssignedTo dgs@NITDA.gov.ng, Status Completed, Priority Low, NVERSE DGOFASTTRACK, Category (Archived, Internships (SIWES) or Internships (NYSC)), EntityKey and IdState ACTIVE, and answers its ID. set-reference-id reads that queue row back and refuses (409) one that is not this activity's DGOFASTTRACK row of that category, then sets Reference_ID to yyyyMMdd(tomorrow, WAT)--<UNC | INT-SIWES | INT-NYSC>-, computed here. update-activity marks the activity Treated, AssignmentStatus Processed (the list offers Processed and Pending; Canvas wrote Assigned), AssignedTo dgs@NITDA.gov.ng and the category. Each step writes an audit event carrying its result, so a repeated request returns the first result (T11). The caller's role must allow the activities route.

- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

Steps added: `Compose_Dynamic_Replay_Meta`, `Compose_Dynamic_Lifecycle`, `Get_Dynamic_Lifecycle_Activity`, `Switch_Dynamic_Lifecycle_Step`, `Compose_Result_Dynamic_404_Lifecycle`, `Create_Dynamic_Lifecycle_Queue`, `Compose_Dynamic_Lifecycle_Create_Data`, `Create_Dynamic_Lifecycle_Create_Audit`, `Compose_Result_Dynamic_200_Lifecycle_Create`, `Compose_Result_Dynamic_422_Lifecycle_Create`, `Get_Dynamic_Lifecycle_Queue`, `Compose_Dynamic_Lifecycle_Reference`, `Condition_Dynamic_Lifecycle_Queue_Matches`, `Update_Dynamic_Lifecycle_Reference`, `Compose_Dynamic_Lifecycle_Reference_Data`, `Create_Dynamic_Lifecycle_Reference_Audit`, `Compose_Result_Dynamic_200_Lifecycle_Reference`, `Compose_Result_Dynamic_409_Lifecycle_Queue`, `Update_Dynamic_Lifecycle_Activity`, `Compose_Dynamic_Lifecycle_Update_Data`, `Create_Dynamic_Lifecycle_Update_Audit`, `Compose_Result_Dynamic_200_Lifecycle_Update`, `Compose_Result_Dynamic_422_Lifecycle_Update`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`, `Compose_Dynamic_Actor_Key`, `Compose_Dynamic_Hash`, `Get_Dynamic_Gate_Proof_Mode`, `Compose_Dynamic_Gate_Proof_Token`, `Get_Dynamic_Gate_Proof`, `Compose_Dynamic_Gate_Identity`, `Compose_Dynamic_Gate_Caller`, `Compose_Invalid_RMD`, `Create_Send_Server_Error_Alert_Email_Receipt`, `Compose_Send_Server_Error_Alert_Email_Outcome`.

Steps removed: `Get_Dynamic_Document`, `Compose_Dynamic_Document_State`, `Switch_Dynamic_Document_State`, `Update_Dynamic_Flag`, `Compose_Dynamic_Flag_Meta`, `Create_Dynamic_Flag_Audit`, `Compose_Result_Dynamic_200_Flag`, `Compose_Result_Dynamic_422_Flag`, `Compose_Result_Dynamic_404_Document`, `Compose_Result_Dynamic_409_Document`, `Get_Dynamic_Task`, `Compose_Dynamic_Task_State`, `Switch_Dynamic_Task_State`, `Update_Dynamic_Task`,

`Compose_Dynamic_Task_Meta`, `Create_Dynamic_Task_Audit`, `Compose_Result_Dynamic_200_Task`, `Compose_Result_Dynamic_422_Task`, `Compose_Result_Dynamic_404_Task`, `Compose_Result_Dynamic_409_Task`, `Get_Dynamic_Status_Document`, `Compose_Dynamic_Status_State`, `Switch_Dynamic_Status_State`, `Condition_Dynamic_Status_Changed`, `Compose_Result_Dynamic_200_Status_Unchanged`, `Update_Dynamic_Status`, `Compose_Dynamic_Status_Meta`, `Create_Dynamic_Status_Audit`, `Compose_Result_Dynamic_200_Status`, `Compose_Result_Dynamic_422_Status`, `Compose_Result_Dynamic_404_Status`, `Compose_Result_Dynamic_409_Status`, `Compose_Dynamic_Event_Meta`, `Create_Dynamic_Event_Audit`, `Compose_Result_Dynamic_200_Event`, `Compose_Result_Dynamic_503_Event`.

Test after pasting. In Folio, Activities: open an activity and choose Route to SIWES. Global Tracking Queue has a new row NVERSE DGOFASTTRACK, Category Internships (SIWES), Reference_ID yyyyMMdd--INT-SIWES-; the activity shows Status Treated and AssignmentStatus Processed; DGO_AuditLog has activity:queued, activity:reference-set and activity:treated. Archive (UNC) and NYSC likewise.

EMAIL_RELATED_TASK

- Folio's request reaches the flow: core/data-client.js sends {action, payload:{...}, userEmail, requestId}, with the e-mail as payload.email and the task fields flat in payload (core/assignment-payload.js buildEmailTaskPayload), while every step of the flow read Email.id and NewActivityTask.* at the top level. Folio's requests therefore carried no e-mail id and were refused at validation. Compose_Email_Task_Input, the scope's first step, gives the steps the fields they read, from the top-level form when a request carries it and from Folio's payload otherwise (priority capitalised and status New, as the Assignment Desk sends them), and every step reads it instead of the trigger.
- S8 / ID-19: every run mints its Request ID. varRequestId took the client's requestId whenever the request carried one (Folio always sends one), so the reply, the audit and every row written carried a value the client chose. The scope now sets it to guid() before anything reads it.
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from the request (userEmail, else Folio's createdBy). With the switch true, a caller who is not an active directory user is refused with 401 and one whose role does not allow single-assignment (the Folio route that owns create-task-from-email, config/action-ownership.config.js) with 403, through the flow's own validation-failure path, before any number is claimed or any row written. The task's AssignedBy is that caller.
- B1: the audit e-mail's attachment list is built without a Filter array, which a pasted scope may not hold. Select_Audit_Attachments turns each attachment with a name and content into its JSON text and Compose_Audit_Attachments joins them into the same list Filter_array_Compose_Data_FlowRun_Verification_Email_Attachments gave.
- S3 / §8.2: each task gets a Task Reference TASK-YYYY-NNNNN, claimed by the generation kit (10 flat attempts, IF-MATCH, counter row TASK- created by the first claim of the year), addressed through Flow Configuration (DATASET_SEQUENCE_COUNTERS_*). If no number can

be claimed the task is not created and the run takes the existing Task Processing catch, which records the error.

- §7.1 / §15: the task names its e-mail's record when the e-mail is registered. The e-mail's internetMessageId (Get_Email) is looked up in DGO_IdentityMap as an EXO_INTERNET_MESSAGE_ID LOCATOR, which DYNAMIC_ACTIONS register writes (T8); when it points at a REC, the task's ParentKey and RootKey are that REC and RootRef its NITDA-YYYY-NNNNN from Portal Registry. An e-mail no one has registered gives an empty ParentKey, and the task still keeps the message id in ActivityTrackingID and RefIDD, as before. No map entry is written here: an e-mail's locator belongs to its record (§7.1), and one written for a task would block the e-mail's registration. A failed lookup does not stop the task.
- §10.2 TASK_QUEUE: the task is created in Global Tracking Queue (ee82725a-c408-45e2-a8e8-facf7a092047), the list Folio reads tasks from (FETCH_ALL) and Single and Bulk Assignment write. The flow created it in DEV_OPS_Global_Tracking_Queue (22c4e38d-5040-4e0f-aa47-dd3aa3f28abf), a test list (owner direction, 5 October 2026; spec 1.2.4), so a task raised from an e-mail never appeared in Folio, and then Update_item_Task_Email wrote the reference and task type into Global Tracking Queue at the test list's item ID — onto whichever unrelated task holds that ID there. The create now carries the task type and grouping itself, the Entity Key, TaskRef, ParentKey, RootKey, RootRef, IdState ACTIVE, Rev 1, the Request ID as CorrelationId and CreatedByRequest, ClientRequestHash (B3), and Reference_ID, the legacy reference column, holding the Task Reference until Phase 6; the update is removed (B9).
- The task's DueDate is written. It was null unless the due date was given as a non-text value, which JSON never does, and its expression called type(), which is not a workflow expression function, so the step could not evaluate it. DueDate is now the given due date (TaskDueDate, else TaskDue) in yyyy-MM-ddTHH:mm:ssZ, or empty when none is given.
- StartDate and the acknowledgement due date no longer fail when the request gives none. Each chose between a default and formatDateTime(), and an expression evaluates both branches of if(), so an absent date stopped the create. The given date is now read as 1900-01-01 where it is empty, in the branch that is then not taken; the defaults are unchanged.
- S3: new tasks have no composite reference. Compose_ReferenceID (-EML---), which the e-mails and the reply show, is now the Task Reference, and the acknowledgement and support links' ref= is the Task Reference instead of TASK- (support.html?ref=TASK-YYYY-NNNNN, §5 ID-05); the links still find the task by item=. The reply's data carries taskRef and entityKey.
- ITEM-50 (owner decision, 2026-10-06): the two task notifications go to the assigned officer, the task row's AssignedTo (the request's assignee when the row has none), with the third-assignee Cc kept. They went to one named address whoever the task was for.
- ITEM-50 / ITEM-52 (owner decision, 2026-10-06): the diagnostics mail (Send_an_email_(V2)*Main_1_1, the raw response to one named address) and the two audit mails (Send_an_email*(V2)_Audit and _Audit_1, which repeated what the audit lists hold) are removed, with Scope_Build_Audit_information, which built nothing else. The reply to the caller is unchanged.
- Deep links: task e-mail links start at https://folio-platform.page.gd (the Workspace's primary host, owner 2026-10-08), not https://taskack.page.gd.

- Tenant defect 2026-10-12: a task raised from an e-mail sent the assignee two near-identical notices (Send_an_email_(V2)*Main_1_1_1, then Scope_3's Send_an_email*(V2)_Main_1_1_2, the same task and links). Scope_3 is removed, so one task sends one notice; the notification count follows the one send.
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_varData_Success, Set_variable_varData_Task_Processing_Catch, Set_variable_varEmailId, Set_variable_varRequestId, Set_variable_varStatusCode.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_an_email_(V2)*Main_1_1_1): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose*_Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.
- ITEM-77 (owner decision, 2026-10-06): every Office 365 step signs in as the registry mailbox, dgsregistry@nitda.gov.ng, connection c0b9e7a5 (Send_an_email_(V2)_Main_1_1_1, Get_Email used another connection).

Steps added: `Compose_Value_Set_variable_varStatusCode`, `Create_Send_an_email_(V2)_Main_1_1_1_Receipt`, `Compose_Send_an_email_(V2)_Main_1_1_1_Outcome`, `Compose_Value_Set_variable_varData_Task_Processing_Catch`, `Get_Dataset_Bindings`, `Select_Dataset_Bindings`, `Compose_Dataset_Bindings`, `Compose_Environment_Prefix`, `Compose_EntityKey_Task`, `Compose_Claim_Task_Year`, `Compose_Claim_Task_Count`, `Scope_Claim_Task_1`, `Get_Counter_Task_1`, `Condition_Counter_Task_1_Exists`, `Compose_Next_Task_1`, `Claim_Counter_Task_1`, `Compose_Claimed_Task_1`, `Create_Counter_Task_1`, `Compose_Created_Task_1`, `Scope_Claim_Task_2`, `Delay_Claim_Task_2`, `Get_Counter_Task_2`, `Condition_Counter_Task_2_Exists`, `Compose_Next_Task_2`, `Claim_Counter_Task_2`, `Compose_Claimed_Task_2`, `Create_Counter_Task_2`, `Compose_Created_Task_2`, `Scope_Claim_Task_3`, `Delay_Claim_Task_3`, `Get_Counter_Task_3`, `Condition_Counter_Task_3_Exists`, `Compose_Next_Task_3`, `Claim_Counter_Task_3`, `Compose_Claimed_Task_3`, `Create_Counter_Task_3`, `Compose_Created_Task_3`, `Scope_Claim_Task_4`, `Delay_Claim_Task_4`, `Get_Counter_Task_4`, `Condition_Counter_Task_4_Exists`, `Compose_Next_Task_4`, `Claim_Counter_Task_4`, `Compose_Claimed_Task_4`, `Create_Counter_Task_4`, `Compose_Created_Task_4`, `Scope_Claim_Task_5`, `Delay_Claim_Task_5`, `Get_Counter_Task_5`, `Condition_Counter_Task_5_Exists`, `Compose_Next_Task_5`, `Claim_Counter_Task_5`, `Compose_Claimed_Task_5`, `Create_Counter_Task_5`, `Compose_Created_Task_5`, `Scope_Claim_Task_6`, `Delay_Claim_Task_6`, `Get_Counter_Task_6`, `Condition_Counter_Task_6_Exists`, `Compose_Next_Task_6`, `Claim_Counter_Task_6`, `Compose_Claimed_Task_6`, `Create_Counter_Task_6`, `Compose_Created_Task_6`, `Scope_Claim_Task_7`, `Delay_Claim_Task_7`, `Get_Counter_Task_7`, `Condition_Counter_Task_7_Exists`, `Compose_Next_Task_7`, `Claim_Counter_Task_7`, `Compose_Claimed_Task_7`, `Create_Counter_Task_7`, `Compose_Created_Task_7`, `Scope_Claim_Task_8`, `Delay_Claim_Task_8`, `Get_Counter_Task_8`, `Condition_Counter_Task_8_Exists`, `Compose_Next_Task_8`, `Claim_Counter_Task_8`, `Compose_Claimed_Task_8`, `Create_Counter_Task_8`, `Compose_Created_Task_8`, `Scope_Claim_Task_9`, `Delay_Claim_Task_9`, `Get_Counter_Task_9`, `Condition_Counter_Task_9_Exists`, `Compose_Next_Task_9`, `Claim_Counter_Task_9`, `Compose_Claimed_Task_9`, `Create_Counter_Task_9`,

`Compose_Created_Task_9`, `Scope_Claim_Task_10`, `Delay_Claim_Task_10`, `Get_Counter_Task_10`, `Condition_Counter_Task_10_Exists`, `Compose_Next_Task_10`, `Claim_Counter_Task_10`, `Compose_Claimed_Task_10`, `Create_Counter_Task_10`, `Compose_Created_Task_10`, `Compose_Claim_Task`, `Select_Claim_Task_References`, `Compose_Task_Claim_Check`, `Compose_Task_Ref`, `Get_Email_Locator`, `Compose_Email_Record_Key`, `Get_Email_Record`, `Compose_Email_Record_Ref`, `Compose_Value_Set_variable_varEmailId`, `Compose_Value_Set_variable_varData_Success`, `Compose_Email_Task_Input`, `Set_variable_varRequestId`, `Get_Assign_Gate_Proof_Mode`, `Compose_Assign_Gate_Proof_Token`, `Get_Assign_Gate_Proof`, `Compose_Assign_Gate_Identity`, `Compose_Assign_Gate_Caller`, `Get_Assign_Gate_Directory_User`, `Compose_Assign_Gate_Role`, `Get_Assign_Gate_Role_Permissions`, `Compose_Assign_Gate_Auth`, `Compose_Value_Set_variable_varRequestId`.

Steps removed: `Scope_Build_Audit_information`, `Select_SourceEmail`, `Select_AllTasks_Audit`, `Select_Errors_Audit`, `Table_Errors_Audit`, `Table_SourceEmail`, `Table_AllTasks_Audit`, `Compose_Trigger_Body_1`, `Compose_SourceEmailAuditRecord`, `Compose_AuditObject`, `Compose_Trigger_Body_Audit`, `Compose_AuditHtml`, `Filter_array_Compose_Data_FlowRun_Verification_Email_Attachments`, `Send_an_email_(V2)_Audit`, `Compose_Data_FlowRun_Verification_Email_Attachments`, `Send_an_email_(V2)_Audit_1`, `Send_an_email_(V2)_Main_1_1`, `Scope_3`, `Compose_Normalized_Priority`, `Compose_Priority_TextColor`, `Compose_Priority_BgColor`, `Compose_Priority_BorderColor`, `Compose_Priority_BorderColor_Primary`, `Compose_Email_Notification`, `Send_an_email_(V2)_Main_1_1_2`, `Compose_Status_BorderColor`, `Compose_Status_BorderColor_Primary`, `Compose_Status_BgColor`, `Compose_StatusText`, `Compose_Status_TextColor`, `Compose_AttachmentLink`, `Compose_Normalized_Status`, `Compose_Priority_Icon_SVG`, `Compose_Status_Hero_SVG`, `Compose_Header_Band_SVG`, `Compose_Attachment_Icon`, `Compose_Priority_Color`, `Update_item_Task_Email`.

How to apply. Paste over Scope_Global in IP_Create__Task__for_received_Email_Endpoint (ab1a3555), the adopted email-to-task flow. Point Folio's EMAIL_RELATED_TASK URL at this flow if it is not already, then turn off the retired copies 3f2f649b and 2fbb5485 (reference/FLOW_ADOPTION.md).

Test after pasting. In Folio, open an e-mail in Lookup and raise a task from it. In Global Tracking Queue (not DEV_OPS_Global_Tracking_Queue) a new row has the e-mail's subject or the title given, TaskRef = Reference_ID = the next TASK-YYYY-NNNNN, EntityKey, IdState ACTIVE, Rev 1, TaskTypeChoice Standard Task and ActivityTrackingID = the e-mail's id; the officer's e-mail shows the TASK reference; the reply carries data.taskRef and a requestId of the flow's own. Register the same e-mail first through DYNAMIC_ACTIONS register (locatorKind EXO_INTERNET_MESSAGE_ID) and raise another task: that task's ParentKey and RootKey are REC:<the record's key> and RootRef its NITDA-YYYY-NNNNN.

OTP_VERIFY

- Tenant defect (8 October 2026): a verify for an address with no code issued answered 500, because Update_item_Attempts had no row to count against (id empty). The attempt count is

now written only when an unverified code exists; otherwise the flow goes straight to its no-pending-code answer, as a wrong code does, and nothing is written.

- The response data is an object. The flow built it in varResponse, a string, and returned that string as data, so Folio (core/contracts.js liftEnvelope) could not read any field of it, the proof included. Set_variable_varData now parses it; every value the flow appends to varResponse is one JSON object.
- G-04, the mint: when the code verifies, the caller is looked up in DGO_UserDirectory (active rows only). An active directory user gets a proof: a guid() token written to the proof ledger (Purpose internal-session, unconsumed) with an expiry of 4 hours (AUTHENTICATION_CONTRACT.md §2a), and the reply carries token, expiresAt (the same Compose, so the ledger row and the reply agree) and claims read from the directory row. A verified address that is not an active directory user gets 401 and no proof. This is DGO_OTP_VERIFY's mint (retired 2026-10-06), placed in the live flow.
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_Verification_Not_A_Directory_User, Set_variable_varData.
- ITEM-82: Case_Generate leaves IP_OTP_VERIFY. Owner decision 2026-10-07: generation and verification run on separate flows; a Folio sign-in on 2026-10-08 ran generation in IP_OTP_GENERATE (05:14:59Z) and verification here (05:16:15Z, proof created), so this flow now only verifies. A generate request answers 400 from the Switch default.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.

Steps added: `Get_Otp_Directory_User`, `Compose_Otp_Proof_Token`, `Compose_Otp_Proof_Expires`, `Condition_Otp_Known_User`, `Create_Otp_Proof`, `Set_variable_Verification_Not_A_Directory_User`, `Append_to_string_variable_Verification_Not_A_Directory_User`, `Compose_Value_Set_variable_Verification_Not_A_Directory_User`, `Condition_Otp_Has_Pending_Code`, `Compose_Value_Set_variable_varData`.

Steps removed: `Compose_OTP_Expiry`, `Create_item_OTP_Record`, `Append_to_string_variable_varResponse_Generate_OTP`, `Set_variable_varStatusCode_Generate_OTP`, `Send_an_email_(V2)`.

How to apply. Paste over Scope_Global in IP_OTP_VERIFY (4dbe6dde), the one flow OTP_GENERATE and OTP_VERIFY run on. The proof ledger (Global_Digital_Documents_Centre, f5c9698e) already exists. Set Flow Configuration IDENTITY_PROOF_REQUIRED to true only after a sign-in has returned a proof.

Test after pasting. Sign in to Folio with an emailed code as an active directory user. The reply's data carries token, expiresAt (four hours ahead) and claims with the directory row's role; the proof ledger has a row with that token as Title, Purpose internal-session, Consumed No and the same expiry; the next Folio call sends Authorization: Bearer . Sign in with a correct code as an address that is not in DGO_UserDirectory: 401 NOT_A_DIRECTORY_USER and no ledger row.

OTP_GENERATE

- Owner decision, 2026-10-07: OTP_GENERATE has a flow of its own. This is the OTP_VERIFY package with Case_Verify removed: the code is created in OTP_Transactions and e-mailed exactly as IP_OTP_VERIFY does it; a verify request answers 400 from the Switch default.
- Codes go only to active directory users: Case_Generate looks the address up in DGO_UserDirectory (active rows) and creates and mails the code only when it is found. The reply (200, OTP sent, the expiry) is the same for any address, so the endpoint does not reveal who is a user; IP_OTP_VERIFY still refuses a proof to anyone outside the directory.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_an_email_(V2)): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

The scope is new: it replaces the flow's Scope_Global as a whole (29 steps).

How to apply. Created in the tenant by the Flow API (2026-10-07). Point Folio's OTP_GENERATE URL at this flow; once a sign-in has run through it, Case_Generate leaves IP_OTP_VERIFY.

Test after pasting. Sign in to Folio: the code arrives from dgsregistry@nitda.gov.ng, and the run appears in IP_OTP_GENERATE's history (not IP_OTP_VERIFY's); OTP_Transactions has the new row; Portal Outbox Receipts has a row with Status sent. Entering the code verifies through IP_OTP_VERIFY as before.

REFERENCE_DATA

- S8 / ID-19: the run sets its own Request ID (varRequestId = guid()) first. The flow declared it from the client's correlation id or requestId when one was sent.
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. The role is read from the directory row's Role choice, which the flow read as text and so never matched a RoleId.
- A caller with no role-permission row made the read fail with 500: Compose_Lookups_Permissions_Array and Compose_Lookups_Routes_Array chose between json('[]') and json(<the row's text>) with if(), which evaluates both, and json('') throws. They now parse one string, '[]' when the row is absent or empty. With the switch true this is what lets a caller with no proof be answered 401 rather than 500.
- The reference data endpoint answered anyone: Condition_Lookups_Authorised read Compose_temp_fix_permtted and Compose_temp_fix_caller, which hold the literal true, in place of the caller and permission check the flow computes. Those steps (and Compose_actual_expressions_Condition_Lookups_Authorised, which only displayed the real values) are removed, and the condition decides on the caller: while

IDENTITY_PROOF_REQUIRED is not true it passes as today; when it is true, the caller must be an active DGO_UserDirectory user resolved from a valid proof, else 401 UNAUTHENTICATED. The role's route and permission (Compose_Lookups_Permitted) are still not enforced: which roles hold them is not recorded in this repository, and enforcing them unverified would lock officers out.

- Directory and reference data as Folio reads them, as FETCH_ALL's package reads them (ported from the retired DGO_REFERENCE_DATA design, registers/flow-retirement.json): the directory is read in pages to 5,000 rows (it holds 852; the flow read one page of 500), Role, Persona and Status are served as text rather than Choice objects, and Departments and Categories, imported from spreadsheets so GetItems returns field_1...field_N, are read by those internal names. The keys Folio reads are unchanged.
- The full directory only to a holder of user:view (or of every route, "*"), as the retired DGO_REFERENCE_DATA design served it. Every caller gets every active user's UserId, FullName, Email, Directorate and Department, which the Workspace needs to address and assign; Role, Persona, Status, AccessScope and PilotCohort are served for the caller's own row and, to anyone else, as null. The rows and their keys are unchanged; one Select does it, so no Filter array is pasted.
- ITEM-52: the Response_Lookups run record carries the reply's size instead of the reply, and the read endpoint no longer e-mails its run record (owner decision 2026-10-06; tenant evidence 2026-10-10).
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_Lookups_Catch_Data, Set_Lookups_Denied_Data, Set_Lookups_Denied_Status, Set_Lookups_Success_Data, Set_variable_varRequestId.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.

Steps added: `Compose_Lookups_Sees_Directory`, `Compose_Value_Set_Lookups_Success_Data`, `Compose_Value_Set_Lookups_Denied_Data`, `Compose_Value_Set_Lookups_Denied_Status`, `Get_Lookups_Gate_Proof_Mode`, `Compose_Lookups_Gate_Proof_Token`, `Get_Lookups_Gate_Proof`, `Compose_Lookups_Gate_Identity`, `Compose_Lookups_Gate_Caller`, `Compose_Value_Set_Lookups_Catch_Data`, `Set_variable_varRequestId`, `Compose_Value_Set_variable_varRequestId`.

Steps removed: `Compose_temp_fix_permtted`, `Compose_temp_fix_caller`, `Compose_actual_expressions_Condition_Lookups_Authorised`, `Compose_Telemetry_Attachments`, `Send_Telemetry_Email`.

How to apply. Paste over Scope_Global in IP_Reference_Data_Endpoint (9a4dea8e). SharePoint connections arrive bound. Paste before IDENTITY_PROOF_REQUIRED is set to true.

Test after pasting. Open Folio as an officer whose role holds user:view: the directory lists every active user (more than 500 if the directory holds them) with Role, Persona and Status as text, and the category and department pickers show codes and heads. Open it as an officer whose role does

not: every user is listed with name, e-mail, directorate and department, Role and the rest are blank except on the officer's own row. With IDENTITY_PROOF_REQUIRED true, call REFERENCE_DATA with no Authorization header: 401 UNAUTHENTICATED.

GET_DOCS

- S8 / ID-19: the run sets its own Request ID (varRequestId = guid()) first. The flow declared it from the client's correlation id or requestId when one was sent.
- G-04: the caller is resolved as every identification package resolves it (see "Caller identity" above): from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. The role is read from the directory row's Role choice, which the flow read as text and so never matched a RoleId.
- A caller with no role-permission row made the read fail with 500: Compose_GetDocs_Permissions_Array and Compose_GetDocs_Routes_Array chose between json('[]') and json(<the row's text>) with if(), which evaluates both, and json('') throws. They now parse one string, '[]' when the row is absent or empty. With the switch true this is what lets a caller with no proof be answered 401 rather than 500.
- The documents endpoint (GET_DOCS and FETCH_ACTIVITIES) answered anyone: Condition_GetDocs_Authorised read Compose_temp_fix_permtted and Compose_temp_fix_caller, which hold the literal true, in place of the caller and permission check the flow computes. Those steps (and Compose_actual_condition_expressions, which only displayed the real values) are removed, and the condition decides on the caller: while IDENTITY_PROOF_REQUIRED is not true it passes as today; when it is true, the caller must be an active DGO_UserDirectory user resolved from a valid proof, else 401 UNAUTHENTICATED. The role's route and permission (Compose_GetDocs_Permitted) are still not enforced: which roles hold them is not recorded in this repository, and enforcing them unverified would lock officers out.
- ITEM-52: the Response_GetDocs run record carries the reply's size instead of the reply, and the read endpoint no longer e-mails its run record (owner decision 2026-10-06; tenant evidence 2026-10-10).
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_GetDocs_Catch_Data, Set_GetDocs_Denied_Data, Set_GetDocs_Denied_Status, Set_GetDocs_Success_Data, Set_variable_varRequestId.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.

Steps added: `Compose_Value_Set_GetDocs_Success_Data`, `Compose_Value_Set_GetDocs_Denied_Data`, `Compose_Value_Set_GetDocs_Denied_Status`, `Get_GetDocs_Gate_Proof_Mode`, `Compose_GetDocs_Gate_Proof_Token`, `Get_GetDocs_Gate_Proof`, `Compose_GetDocs_Gate_Identity`, `Compose_GetDocs_Gate_Caller`, `Compose_Value_Set_GetDocs_Catch_Data`, `Set_variable_varRequestId`, `Compose_Value_Set_variable_varRequestId`.

Steps removed: `Compose_temp_fix_permtted`, `Compose_temp_fix_caller`, `Compose_actual_condition_expressions`, `Compose_Telemetry_Attachments`, `Send_Telemetry_Email`.

How to apply. Paste over Scope_Global in IP_Get_Docs_Endpoint (28f6e62a); it serves GET_DOCS and FETCH_ACTIVITIES. SharePoint connections arrive bound.

Test after pasting. Open a document list in Folio: documents load as before and the reply carries a requestId of the flow's own. With IDENTITY_PROOF_REQUIRED true, call GET_DOCS with no Authorization header: 401 UNAUTHENTICATED; signed in, the documents load.

FETCH_EMAIL_ATTACHMENTS

- G-04: the endpoint had no caller check at all: anyone holding its URL could read the attachments of any message in the mailbox by its id. It now resolves the caller from the sign-in proof (else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail) and looks the caller up in DGO_UserDirectory. While the switch is not true it answers as today; when it is true, a caller who is not an active directory user resolved from a valid proof gets 401 UNAUTHENTICATED and no message is read. The steps are SharePoint reads, so the flow gains a SharePoint connection (the one the other endpoint flows use).
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_Catch_Data, Set_Invalid_Data, Set_Invalid_Outcome, Set_Invalid_Status, Set_Success_Data.

Steps added: `Compose_Value_Set_Success_Data`, `Compose_Value_Set_Invalid_Data`, `Compose_Value_Set_Invalid_Status`, `Compose_Value_Set_Invalid_Outcome`, `Get_Attachments_Gate_Proof_Mode`, `Compose_Attachments_Gate_Proof_Token`, `Get_Attachments_Gate_Proof`, `Compose_Attachments_Gate_Identity`, `Compose_Attachments_Gate_Caller`, `Get_Attachments_Gate_Directory_User`, `Compose_Attachments_Gate_Auth`, `Compose_Value_Set_Catch_Data`.

Steps removed: none.

How to apply. Paste over Scope_Email_Attachment_Endpoint_REBUILT in IP_Retrieve_Email_Attachment_Endpoint (5bc87cf3). The pasted gate reads SharePoint, which this flow did not use: when the designer asks, pick the SharePoint connection the other endpoint flows use (3f1943c5).

Test after pasting. Open an e-mail with attachments in Folio's Lookup: the attachments load as before. With IDENTITY_PROOF_REQUIRED true, call FETCH_EMAIL_ATTACHMENTS with an emailId and no Authorization header: 401 UNAUTHENTICATED and no message read (the run shows Get_email_V2 skipped).

EMAIL

- S8 / ID-19: the run sets its own Request ID (varRequestId = guid()) first. The flow declared it from the client's correlation id or requestId when one was sent.

- G-04: the caller is resolved from the sign-in proof when one is valid; else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail as today. The flow's own directory check then decides: with the switch true and no valid proof the caller is nobody, no directory row matches, and Condition_SendEmail_Known_Caller answers 401 UNAUTHENTICATED; nothing is sent.
- Set variable values held in a Compose step (the designer dropped inline and changed Set variable values on paste; tenant export of IP_Create__Task__for_received_Email_Endpoint, 6 October 2026): Set_variable_varData_SendEmail_Attached, Set_variable_varData_SendEmail_Catch, Set_variable_varData_SendEmail_Denied_401, Set_variable_varData_SendEmail_Duplicate, Set_variable_varData_SendEmail_Invalid, Set_variable_varData_SendEmail_Plain, Set_variable_varRequestId.
- The sign-in proof is redacted from the run record (tenant defect, 8 October 2026): Compose_Redacted_Headers masked Authorization but not X-DGO-Proof, where the proof travels since release 9, so every telemetry row held a usable session token.

Steps added: `Compose_Value_Set_variable_varData_SendEmail_Attached`, `Compose_Value_Set_variable_varData_SendEmail_Plain`, `Compose_Value_Set_variable_varData_SendEmail_Duplicate`, `Compose_Value_Set_variable_varData_SendEmail_Invalid`, `Compose_Value_Set_variable_varData_SendEmail_Denied_401`, `Get_SendEmail_Gate_Proof_Mode`, `Compose_SendEmail_Gate_Proof_Token`, `Get_SendEmail_Gate_Proof`, `Compose_SendEmail_Gate_Identity`, `Compose_SendEmail_Gate_Caller`, `Compose_Value_Set_variable_varData_SendEmail_Catch`, `Set_variable_varRequestId`, `Compose_Value_Set_variable_varRequestId`.

Steps removed: none.

How to apply. Paste over Scope_Send_Email_Flow in IP_SEND_EMAIL (df59a69f). Connections arrive bound.

Test after pasting. Send a message from Folio: it is delivered and the receipt row is written as before; the reply carries a requestId of the flow's own. With IDENTITY_PROOF_REQUIRED true, call EMAIL with a userEmail in the body and no Authorization header: 401 UNAUTHENTICATED and nothing is sent.

AI

- G-04: AI_Document_Processing (AI_EMAIL_ANALYSIS, AI_DOC_ANALYSIS, AI_CHAT) had no caller check: it read any DGO DIGITAL OPS item's attachments and sent them to an external model for anyone holding its URL. Its steps sit at the top level of the flow, so this is an added scope, not a replacement: it resolves the caller from the sign-in proof (else, while IDENTITY_PROOF_REQUIRED is not true, from userEmail) and looks the caller up in DGO_UserDirectory. While the switch is not true it does nothing visible; when it is true, a caller who is not an active directory user resolved from a valid proof gets 401 UNAUTHENTICATED and the run stops before any item is read.

- Security (ITEM-75): the flow's HTTP action sends an OpenRouter key as literal text in its Authorization header, so every copy of the definition carries it. This scope reads the key from Flow Configuration (OPENROUTER_API_KEY) and composes the header value, Compose_OpenRouter_Authorization = "Bearer ". After pasting, set the HTTP action's Authorization header to @{outputs('Compose_OpenRouter_Authorization')}: that one field is the only edit outside this scope, and afterwards the definition holds no key. With no row the value is "Bearer " and the AI call is refused by OpenRouter rather than sent with the old key.

The scope is new: it replaces the flow's Scope_AI_Caller_Gate as a whole (12 steps).

How to apply. First add a Flow Configuration row OPENROUTER_API_KEY holding a new OpenRouter key (ITEM-75). This scope is added, not replaced: in AI_Document_Processing (fefa43c6) select the + between Initialize_variable_2 and Get_item, choose Paste an action, paste the file. Then open the HTTP action and set its Authorization header to @{outputs('Compose_OpenRouter_Authorization')}, replacing the key text. Save, then revoke the old key in OpenRouter.

Test after pasting. Run an AI analysis from Folio signed in: it answers as before, and in the run history the HTTP action's Authorization shows the configured key, while Export (or Peek code on HTTP) shows only the expression. With IDENTITY_PROOF_REQUIRED true, call AI_DOC_ANALYSIS with no Authorization header: 401 UNAUTHENTICATED, and the run is Cancelled at Terminate_AI_Gate with Get_item not run.

SCAN_PROCESSING

- Trigger: the OneDrive trigger reports a file's properties, not its content (When a file is created (properties only), split per file). Get_Trigger_Document_Content reads the content first, and every reference to the old trigger reads the new one: the content from Get_Trigger_Document_Content, the file id and name from the trigger's Id and Name, the folder from its Path.
- One account: the legacy flow spread its work over three OneDrive and two SharePoint connections, some signed in as other people. Every OneDrive step uses the flow's own OneDrive connection (dgsRegistry@nitda.gov.ng) and every SharePoint step the registry's SharePoint connection (3f1943c5).
- No copy to another person's OneDrive: the legacy flow copied each document to "/2024 DOCUMENTS/Documente Volume II" in a second account and renamed that copy. The document stays where it arrived and is renamed there (Rename_Trigger_File*); CREATE_COPY_OF_TRIGGER_DOCUMENT becomes Get_Trigger_Document_Metadata, which reads the same Id, Name and DisplayName of the file itself, and the steps that moved the copy are removed.
- No flow definition by e-mail: Scope_Flow_Definition_Email read the flow's own definition and mailed it on every run (ITEM-52's surplus pattern). It is removed with its Flow management connection; nothing else read it.
- Deep links: APP_FAST_TRACK_LINK points at the internal platform, not the retired Power Apps player, and Set_variable_Deeplink / Set_variable_Deeplink_2 lose the stray "veggie " suffix the

legacy flow appended to every link (the fix npm run deeplinks:patches gave for the legacy flow).

- Security: the flow held three Google AI API keys as literal text (Compose_API_Key, Compose_API_Key_2 and Compose_API_Key_kanihamza@gmail.com__v2), so every copy of its definition carried them, this repository's capture included. They are now read from Flow Configuration: GEMINI_API_KEY, and GEMINI_API_KEY_2 for the second key the text extraction uses (GEMINI_API_KEY when that row is absent). The step names are unchanged, so the AI calls that read them need no change. The keys that were in the definition are exposed and are to be revoked in Google Cloud and replaced (readiness register).
- Placement: the flow's six variable declarations stay at the top level (Initialise_All_Text_Variable, Initialize_variable_Key_Points, Initialize_variableDeepLink, Initialize_variable_Document_Link, Initialise_varResponse, Initialize_variable_New_File_Name); the rest of the legacy flow is one scope, Scope_Scan_Processing, which starts by reading the document's content.
- S7 / §8.2: every document the library intake takes in becomes a registered record, as a portal submission or a scan does. After the DGO DIGITAL OPS row is created, the scope claims the next NITDA-YYYY-NNNNN (the kit claim, floored at the register's highest number) and writes the Portal Registry row: Title and ReferenceId, Subject the file name, Channel library, CorrespondenceType Incoming, SenderEmail the registry mailbox (Flow Configuration NOTIFY_REGISTRY_TO, else dgsregistry@nitda.gov.ng, as for scans), Status received, AttachmentCount 1, EntityKey, IdState ACTIVE, Rev 1, Direction In, CorrelationId and CreatedByRequest, and SourceLocator spo:/.
- §10.2 OPS_WORKING_REGISTER: the DGO DIGITAL OPS row gets the record's EntityKey, ReferenceId and IdState ACTIVE by the kit's Guarded Update (IF-MATCH; the list has no Rev column). DGO_IdentityMap gets the row as the record's SPO_ITEM locator: Container the DGO DIGITAL OPS list, Value the item ID, MatchKey LOCATOR|SPO_ITEM||. If the register row cannot be written, a VOID row accounts for the number (Q2) and the DGO DIGITAL OPS row is left without a key. If no number can be claimed, nothing is written. Either way the scope ends Succeeded, so the AI processing that follows runs as before.
- S7 / §8.2: Scope_Library_Identity (the registry record, its NITDA-YYYY-NNNNN and its DGO_IdentityMap locator, as designed for the legacy flow) runs after Preliminaries_and_Backup has created the DGO DIGITAL OPS row; Scope_Processing runs after it, whatever it ends.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_an_email_(V2)): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

The scope is new: it replaces the flow's Scope_Scan_Processing as a whole (217 steps).

How to apply. Written whole by scripts/write-scan-processing-flow.mjs (trigger on the OneDrive folder /E-DOCUMENT STATION, bound to the OneDrive connection that reaches it, the six variable declarations, this scope), then kept current by npm run flows:update.

Test after pasting. Scan a document to /E-DOCUMENT STATION. Within a minute: DGO DIGITAL OPS has a new row with the file name, the extracted text in Notes and the AI summary in

Description, the document attached and renamed " - -"; Portal Registry has a row with Channel library and the next NITDA-YYYY-NNNNN; dgs@nitda.gov.ng receives "New E-Scan"; Portal Outbox Receipts records it.

SWEEP

- Owner direction 2026-10-10: one e-mail per person, not one per task. The acknowledgement-overdue and task-overdue loops are replaced by a digest: the eligible tasks of both (the same rules: not acknowledged or not completed, a single valid address) are gathered, and each person receives one e-mail listing all their items, at most once a day (a receipt task-digest: *: is written and checked). Until Flow Configuration NOTIFY_SWEEP_RECIPIENTS is staff, every digest goes to the registry (NOTIFY_OPS_TO) with the intended person named, and the closed-support-case notices go there too: nobody outside the registry is written to before commissioning. The working-hours and NOTIFY_ENABLED switches still apply.*
- Tenant defect 2026-10-18 (AUTO_SCHEDULED_SWEEP run 08584093115454153780858604181CU30): a SharePoint date-only column comes back without a zone, and convertFromUtc refused it. Every date this package converts from a list row, a trigger body or a connector reply (5) is read with a Z added when it has none, and an empty one reads as the epoch.
- ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Case_Email, Send_Undeliverable_Alert_Email, Send_Task_Reminder_Email, Send_Server_Error_Alert_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.

The scope is new: it replaces the flow's Scope_ScheduledSweep_Flow as a whole (103 steps).

How to apply. Written by npm run flows:update (the flow is Stopped). Its schedule is set to weekdays at 08:45 WAT by a separate trigger write. Start it only at commissioning, and set Flow Configuration NOTIFY_SWEEP_RECIPIENTS to staff only when staff may receive the digests.

Test after pasting. Run the sweep once by hand while NOTIFY_SWEEP_RECIPIENTS is not staff: the registry receives one e-mail per person with overdue or unacknowledged tasks, each naming that person and listing all their tasks; Portal Outbox Receipts has one task-digest:

: row per person. Run it again the same day: nothing is sent.

FAILURE_WATCH

- *Owner decision 2026-10-18: every failed notice is registered; only some are reported. The watch reads Flow Configuration (NOTIFY_FAILURE_WATCH_ENABLED, NOTIFY_FAILURE_REPORT_GROUPS, NOTIFY_FAILURE_BURST, NOTIFY_FAILURE_REPORT_TO, and the cursor and health rows it writes itself), falling back to config/notification-failure-groups.config.js when a row is absent.*

- *The window: failures registered after the cursor and up to this run's start (Created), so each is read by exactly one run; a run that cannot read leaves the cursor, and the next run reads the window again. Then every failure still open (Status failed), for the counts and the 07:00 summary. Each failure is placed by its receipt's MessageType, or by flow and type where two flows share a type. Only the reference, type, masked recipient, error and time are carried: never a message body.*
- *What is reported: in the 07:00 run, every reportable failure still open (the morning summary); in every other run, the reportable failures registered since the last run, each once. Reportable: the groups in NOTIFY_FAILURE_REPORT_GROUPS (default citizen, unclassified), and any failure the mapping cannot place. A burst (NOTIFY_FAILURE_BURST or more failures of any group in the window, default 5) is reported as a broken mail connection. Nothing is sent when there is nothing to report, or when NOTIFY_FAILURE_WATCH_ENABLED is false.*
- *Afterwards: the cursor moves to this run's start only when the window was read, and NOTIFY_FAILURE_HEALTH is written (the open count per group, the burst flag, the time) only when the open failures were read. Each row is created on the first run if it is absent.*
- *ITEM-66: every mail send writes a receipt to Portal Outbox Receipts (Send_Failure_Watch_Email): message type, recipient (the send's own To), the run as reference, sent or failed, and the error on failure; never the message body. Compose__Outcome fails exactly when the send failed, so the scope around it ends as it did before the receipt was added and the flow's catch path still fires.*

The scope is new: it replaces the flow's Scope_Failure_Watch as a whole (34 steps).

How to apply. *Created by scripts/write-failure-watch-flow.mjs (a Recurrence trigger: every hour, Monday to Friday, 07:00 to 18:00 WAT; this scope), then kept current by npm run flows:update. Record its id in scripts/lib/flow-artifact-name.mjs FLOW_IDENTITIES and capture it.*

Test after pasting. *Make one notice fail on purpose in each group (a citizen outcome to an address that does not exist, a staff notice, a sign-in code). Each has a Portal Outbox Receipts row with Status failed. At the next hour the registry receives one Folio Failure Watch e-mail listing only the citizen outcome; the next run does not list it again; the 07:00 run lists it while its Status is failed. Five failures in one hour bring the broken-connection line. Flow Configuration NOTIFY_FAILURE_HEALTH holds the open count per group, and the Workspace's System Health shows it.*